408

数据结构

目　录

一、时间复杂度与空间复杂度.....4
二、线性表.....4
(一)线性表的基本概念.....5
(二)线性表的实现.....5
1.顺序存储.....5
2.链式存储.....5
(三)线性表的应用.....7
三、栈、队列和数组.....8
(一)栈和队列的基本概念.....8
(二)栈和队列的顺序存储结构.....8
(三)栈和队列的链式存储结构.....11
(四)多维数组的存储.....12
(五)特殊矩阵的压缩存储.....12
(六)栈、队列和数组的应用.....13
四、树与二叉树.....17
(一)树的基本概念.....17
(二)二叉树.....18
1.二叉树的定义及其主要特征.....18
2.二叉树的顺序存储结构和链式存储结构.....19
3.二叉树的遍历.....20
4.线索二叉树的基本概念和构造.....21
(三)树、森林.....23
1.树的存储结构 23
2.森林与二叉树的转换.....24
3.树和森林的遍历.....24

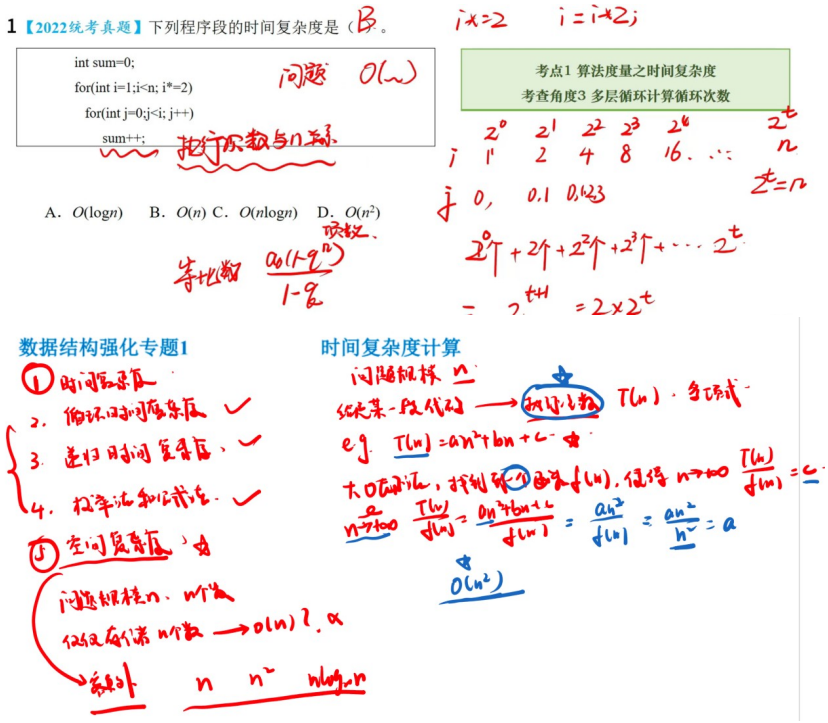
(四)树与二叉树的应用.....25
1.哈夫曼(Huffman)树和哈夫曼编码.....25
2.并查集及其应用.....25
3.堆及其应用.....25
五、图.....26
(一)图的基本概念.....26
(二)图的存储及基本操作.....28
1.邻接矩阵.....28
2.邻接表 29
4.邻接多重表、十字链表.....30
(三)图的遍历.....31
1.广度优先搜索.....31
2.深度优先搜索.....33
(四)图的基本应用.....34
1.最小(代价)生成树.....34
2.最短路径.....36
3.拓扑排序 39
4.关键路径.....40
5.有向无环图 DAG 描述表达式.....43
六、查找.....44
(一)查找的基本概念.....44
(二)顺序查找法.....44
(三)分块查找法 （索引顺序查找）.....44
(四)折半查找法.....45
(五)树型查找.....47

1.二叉树搜索树.....47
2.平衡二叉树.....48
3.红黑树.....51
(六)B 树及其基本操作、B+树的基本概念.....53
(七)散列(Hash)表.....58
(八)字符串模式匹配.....60
(九)查找算法的分析及应用.....60
七、排序.....61
(一)排序的基本概念.....61
(二)直接插入排序.....61
(三)折半插入排序.....62
(四)希尔排序(shell sort).....62
(五)起泡排序(bubble sort).....63
(六)快速排序.....64
(七)简单选择排序.....66
(八)堆排序.....67
(九)二路归并排序(merge sort).....69
(十)基数排序.....71
(十一)计数排序.....72
(十二)外部排序.....73
(十三)排序算法的分析和应用.....78
八、 B 树的插入与删除.....80
(一)B 树的插入.....80
(二)B 树的删除.....80

408——数据结构

一、时间复杂度与空间复杂度

（时间复杂度）——找到步骤执行次数 t 和规模 n 的关系，一般为列举法，找规律即可；



主定理

主定理用于求这类分治递推式的时间复杂度：

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

其中 a 是子问题个数，b 是规模缩小倍数，$f(n)$ 是分解、合并等非递归工作量。

看 ab f (n) ，令 p=logb a
比较 fn 与 n^p
情况 1:fn 大于 n^p ，取 fn
情况 2:fn 等于 n^p ，取 fn 乘以 logn
情况 3:fn 小于 n^p ，取 n^p
SY
2026/08/12 21:22

二、线性表

基本操作	顺序表	链表	静态链表
按位查找	O(1) 顺序表支持随机访问	O(n) 在链表查找第i个元素只能遍历	O(n) 在链表查找第i个元素只能遍历
按值查找	O(n) 需要一个个遍历元素 并与查找元素值对比 才能确定是否查找成功	O(n) 需要一个个遍历元素 并与查找元素值对比 才能确定是否查找成功	O(n) 需要一个个遍历元素 并与查找元素值对比 才能确定是否查找成功
插入	O(n) 需要大量移动元素腾出空位 如果剩余连续空间不够 还需将整个顺序表复制粘贴到 一片新的更大的连续空间	O(n)+O(1) 需要遍历来确定插入位置的前驱节点 然后修改指针进行插入	O(n)+O(1) 需要遍历来确定插入位置的前驱节点 然后修改数组进行插入
删除	O(n) 需要大量移动元素覆盖被删元素	O(n)+O(1) 需要遍历来确定删除位置的前驱节点 然后修改指针进行删除	O(n)+O(1) 需要遍历来确定删除位置的前驱节点 然后修改数组进行删除

按位序从前往后输出，顺序表和链表的时间复杂度是一样的

SY
2026/08/12 21:39

(一)线性表的基本概念

线性表=链表/顺序表

线性表中的元素下标是从 1 开始的，即 a1 、 a2 ，不存在 a0 ！
但是数组中的元素下标是从 a0 开始的
YYK
2026/07/23 16:26

线性表是一种逻辑结构，而链表和顺序表是一种存储结构！
链式存储结构比顺序表更能表示不同的逻辑结构！（虽然顺序表可以表示树或者图，但是明显链式结构表示的更加灵活清楚）

频繁的插入和删除操作，链式结构明显优于顺序结构，因为不需要大面积的移动元素
YYK
2026/07/23 16:36

顺序表

插入用 n-i+1

删除用 n-i

1. 栈：只允许在一端进行插入或删除操作的线性表（操作受限） LIFO

   卡特兰数公式：当n个不同元素进栈时，出栈不同排列个数为 $\frac{1}{n+1}C\binom{n}{2n}$ 也是有n个结点的二叉树的形态的数量

(二)线性表的实现

1.顺序存储

顺序表

顺序表必须物理连续，数组可以不物理连续，并且顺序表和数组在逻辑结构上也可能并不相同！例如树的结构也可以用数组表示

相对于链表来说，顺序表的存储密度更大，因为不需要指针域，所以不需要浪费空间。
YYK
2026/07/23 16:46

顺序表折半查找的时间复杂度为 $\log_2 n$

2.链式存储

（1）单链表

动态分配数组：

1、申请新的数字

2、用循环把旧数据搬过去

3、free 释放旧数组

4、把指针指向新的数组

带头结点与不带头结点

链表的指针 p->data 代表的是*p 这个节点的数据域，同时（*p）.data 代表数据域

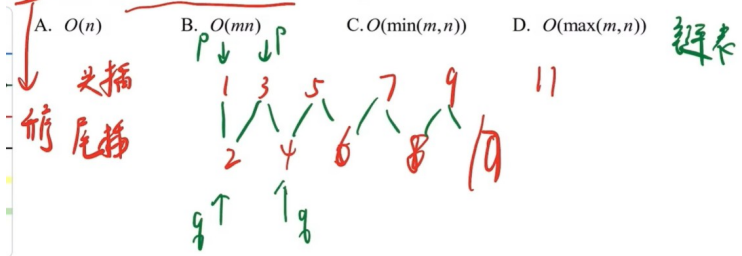
表尾的指针域要用 NULL 表示

头插法、尾插法建立单链表

链表的合并：

9.【2013 统考真题】已知两个长度分别为 m 和 n 的升序链表，若将它们合并为长度为 m+n 的一个降序链表，则最坏情况下的时间复杂度是(　　)。

A. $O(n)$　　B. $O(mn)$　　C. $O(\min(m,n))$　　D. $O(\max(m,n))$



（2）双链表

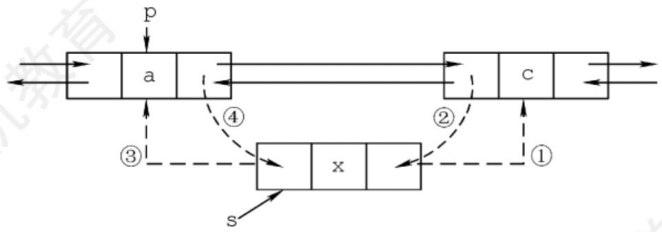


图 2.10　双链表插入结点过程

带头结点的链表一定有头指针（否则找不到头结点），但头指针是指向头结点的独立指针，不是头结点“自带”的。
YYK
2026/07/24 22:47

线性表的长度是不包含头结点的，头指针里可以存放整个表长，加入头指针的目的是为了使得操作更加统一
YYK
2026/07/23 17:03

不带头结点的单链表也是需要头指针的
SY
2026/08/12 21:44

在表头插入一个新结点
不带头结点时，需要特别注意要修改头指针 L
带头结点不需要
SY
2026/08/12 21:38

尾插法的时候，一般都是在链表尾部设置一个指针，这样使得时间复杂度也是 O（n）这点我之前确实没有想到，还以为要从头再次遍历到链尾
YYK
2026/07/24 19:53

非常重要的一个题外知识点，我以为链表每次需要找到最大的一个元素放到最前面，但是实际上，可以采用头插法实现逆序放入元素；即从链表的最开始就可以取走元素，不需要每次都遍历到最后一个元素然后再取出，这是时间复杂度非常高昂的；另外可以延伸到算法题，即将升序的链表逆序的时间复杂度为 O（n）

与上述同题
YYK
2026/10/06 14:51

```
① s->next=p->next;
② p->next->prior=s;
③ s->prior=p;
④ p->next=s;
```

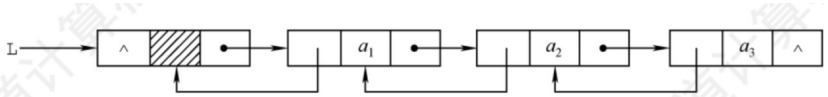


图 2.9　双链表示意图

观察这个图发现，双链表的头节点也在双链表的“循环体”当中，而不是被孤立的
YYK
2026/07/24 19:55

（3）循环链表

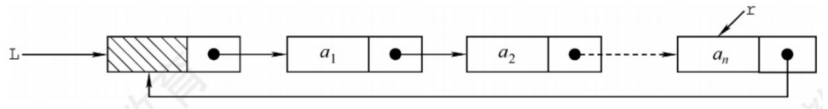


图 2.12　循环单链表

循环链表相对于单链表有两点区别：第一个是在链表尾部的节点的 next 域指向头结点；第二个是在链表尾部的节点增设一个尾指针 r 指向他尾指针

所以判断链表是否为空的依据就是，尾指针 r 它是否等于头指针 L（这个头结点并不是头指针自带的，而是设置的，有时候题目也会设置为 head 指针指向头结点）。 r->next=L ，即 L
YYK
2026/07/24 19:57

尾指针的复原问题，其实刚开始就自行单独开辟一个空间，从尾指针复制一份，进行操作，尾指针不变。
YYK
2026/07/25 14:52

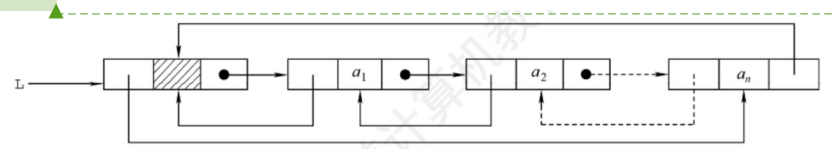


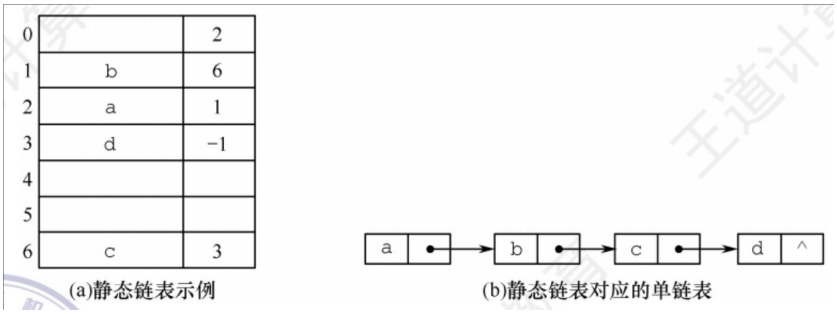
图 2.13　循环双链表

一定要注意，还有尾部指向头结点、头结点指向尾节点这两个！！！
YYK
2026/07/24 21:02

循环双链表为空时，则头结点的 prior 和 next 都为 L 指针

（4）静态链表

在定义时就已经确定了大小、链表的元素个数，不能更改，另外查询工作也不是随机访问，而是需要访问静态链表，按照链表的形式一个一个查询指针链接的下一个元素
YYK
2026/07/24 22:48



(a)静态链表示例　　(b)静态链表对应的单链表

非常巧妙，适用于一些不支持指针的语言

(三)线性表的应用

三、栈、队列和数组

(一)栈和队列的基本概念

卡特兰数

当 n 个不同元素入栈时，出栈元素不同排列的个数为 $\frac{1}{n+1}C_{2n}^{n}$。

这同样也是有 n 个节点的二叉树的形态的数量

YYK
2026/07/25 16:03

应用场景：

1.n个元素进行栈操作（入栈序列确定）有多少种**合法出栈序列**
2.先序序列含n个不同元素时（先序序列确定），能够确定多少种**不同的二叉树**

【2015】先序序列为a，b，c，d的不同二叉树的个数是
A.13
B.14
C.15
D.16

构成不同的二叉树个数也是卡特兰数的经典应用

SY
2026/07/24 17:08

9.元素a,b,c,d,e,f依次进入一个初始为空的栈中，若每个元素入栈后可停留、可出栈，直到所有元素都出栈，则所有可能的出栈序列总数是
A.20
B.42
C.132
D.720

钉死入栈序列，只看出栈序列，就是卡特兰数的经典应用

SY
2026/07/24 17:06

61. 3 对括号，合法的括号序列有（ ）种。
A.3　　B.4　　C.5　　D.6

没想到这题也是卡特兰数。。。

SY
2026/07/24 21:25

栈和队列都具有相同的逻辑结构（线性）

(二)栈和队列的顺序存储结构

栈顶指针：S.top，初始时设置 S.top=-1；栈顶元素：S.data[S.top]。
入栈操作：栈不满时，栈顶指针先加 1，再送值到栈顶。
出栈操作：栈非空时，先取栈顶元素，再将栈顶指针减 1。
栈空条件：S.top==-1；栈满条件：S.top==MaxSize-1；栈长：S.top+1。
另一种常见的方式是：初始设置栈顶指针 S.top=0；入栈时先将值送到栈顶，栈顶指针再加 1；出栈时，栈顶指针先减 1，再取栈顶元素；栈空条件是 S.top==0；栈满条件是 S.top==MaxSize。

两种不同的设置栈指针的方式，第一种是栈顶指针直接指向栈顶元素，第二种是栈顶指针指向栈顶元素的下一个元素。一定要关注栈空栈满的条件

YYK
2026/07/25 14:59

这里判断到底是指针先移动还是先放入元素，你就根据空栈的情况来判断，指针指向 -1，则肯定要先移动指针 i=0 才能装入数据

YYK
2026/07/25 15:10

（2）共享栈

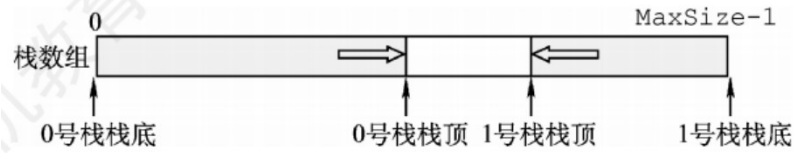


图 3.3　两个顺序栈共享存储空间

栈顶指针指向栈顶元素的前提下，栈顶指针相邻，即差为 1 ，则说明相邻
YYK
2026/07/25 15:04

（3）普通队列

队列的顺序实现是指分配一块连续的存储单元存放队列中的元素，并附设两个指针：队首指针 front 指向队首元素，队尾指针 rear 指向队尾元素的下一个位置（不同教材对 front 和 rear 的定义可能不同，例如，可以让 rear 指向队尾元素、front 指向队首元素。对于不同的定义，出入队的操作是不同的，本节后面有一些相关的习题，读者可以结合习题思考）。

队列的顺序存储类型可描述为

```
#define MaxSize 50              //定义队列中元素的最大个数
   typedef struct{
   ElemType data[MaxSize];      //用数组存放队列元素
   int front,rear;              //队首指针和队尾指针
}SqQueue;
```

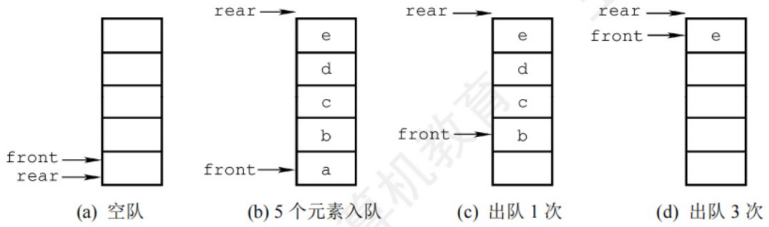
初始时：Q.front=Q.rear=0。

入队操作：队不满时，先送值到队尾元素，再将队尾指针加 1。

出队操作：队不空时，先取队首元素值，再将队首指针加 1。

图 3.6(a)所示为队列的初始状态，有 Q.front==Q.rear==0 成立，该条件可以作为队列判空的条件。

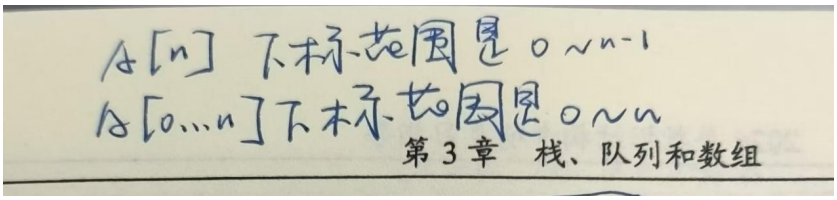
对于普通队列来说，也有两种定义的形式
就是队首指针指向队首元素，rear 指向队尾元素的下一个空位置
Front 指向队首元素， rear 指向队尾元素
YYK
2026/07/25 15:47



(a) 空队　(b) 5 个元素入队　(c) 出队 1 次　(d) 出队 3 次

图 3.6　队列的操作

显然对应的是情况 1
YYK
2026/07/25 15:49



不能用普通的头指针和尾指针相等来判断队列满，有可能是假溢出！

12. 下列描述的几种链表中，最适合用作队列的是（ ）。

A. 带队首指针和队尾指针的循环单链表

B. 带队首指针和队尾指针的非循环单链表

C. 只带队首指针的非循环单链表

不选 A 的原因是因为，这个“循环”的指针，非常多余，所以没必要使用
YYK
2026/07/26 15:19

（4）循环队列

上面指出了顺序队列“假溢出”的问题，这里引出循环队列的概念。将顺序队列臆造为一个环状的空间，即把存储队列元素的表从逻辑上视为一个环，称为循环队列。当队首指针 Q.front=MaxSize-1 后，再前进一个位置就自动到 0，这可以利用除法取模运算（%）来实现。

命题追踪 ▶▶ 特定条件下循环队列队头/队尾指针的初值（2011）

初始时：Q.front=Q.rear=0。
队首指针进 1：Q.front=(Q.front+1)%MaxSize。
队尾指针进 1：Q.rear=(Q.rear+1)%MaxSize。
队列长度：(Q.rear+MaxSize-Q.front)%MaxSize。
出入队时：指针都按顺时针方向进 1（如图 3.7 所示）。

牢牢记住这个插入时的 front 和 rear 的位置关系，有两种情况的，front 指向队头元素、rear 指向队尾元素；或者 front 指向队头元素，rear 指向队尾元素的下一个元素

但记着：front 只会沿着数字增大的方向（除了 maxsize 到 0 这跳跃这一次）
YYK
2026/07/26 11:04

当队空和队满都有 Q.front=Q.rear，所以下列三种方式来修正这个缺陷

①

1）牺牲一个单元来区分队空和队满，入队时少用一个队列单元，这是一种较为普遍的做法，约定以“队首指针在队尾指针的下一位置作为队满的标志”，如图 3.7(d2)所示。
队满条件：(Q.rear+1)%MaxSize==Q.front。
队空条件：Q.front==Q.rear。
队列中元素的个数：(Q.rear-Q.front+MaxSize)%MaxSize。

②数据结构中增设 size 单元记录队列存储数据的数量，若=0 则空，若=maxsize 则满

③数据结构中增设 tag 单元记录队列存储数据的标志，删除成功则置为 0，且导致 Q.front=Q.rear 则队空，插入成功则置为 1，且导致 Q.front=Q.rear 则队满

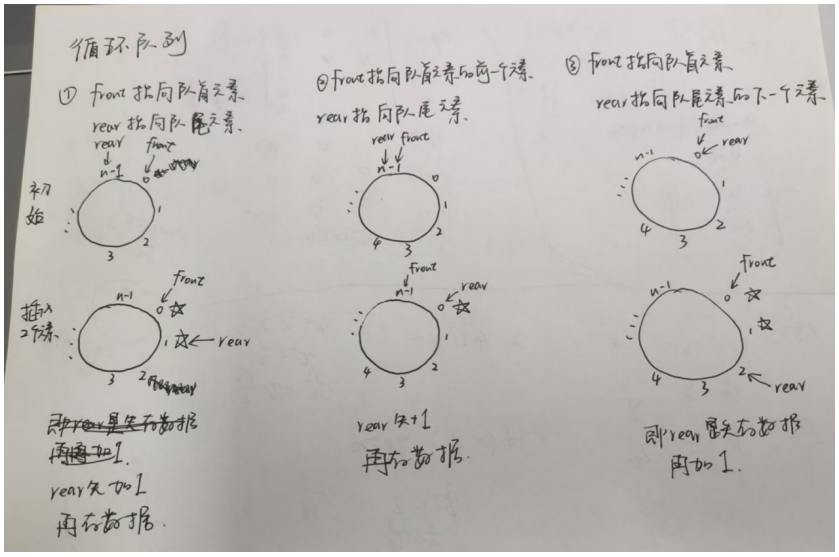
08. 假设用数组 Q[MaxSize]实现循环队列，队首指针 front 指向队首元素的前一位置，队尾指针 rear 指向队尾元素，则判断该队列为空的条件是（ ）。

A. Q.rear==(Q.front+1)%MaxSize
B. (Q.rear+1)%MaxSize==Q.front+1
C. (Q.rear+1)%MaxSize==Q.front
D. Q.rear==Q.front

遇到这种首先要倒推，假设队列中只有一个元素时的情况要符合题目要求，然后倒推这个队列为空的时候，front 和 rear 分别指向谁！（先存入数据，rear 再加 1；还是先 rear 加 1 再存入数据要具体分析）

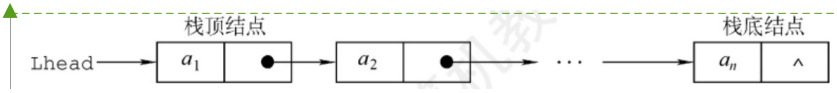
2011、2014 真题
YYK
2026/07/26 15:02

王道计算机教育 www.kaoyan.com



(三)栈和队列的链式存储结构

(1)链式栈



09. 设链表不带头结点且所有操作均在表头进行，则下列最不适合作为链栈的是（ ）。

A. 只有表头结点指针，没有表尾指针的双向循环链表

B. 只有表尾结点指针，没有表头指针的双向循环链表

C. 只有表头结点指针，没有表尾指针的单向循环链表

D. 只有表尾结点指针，没有表头指针的单向循环链表

(2)链式队列

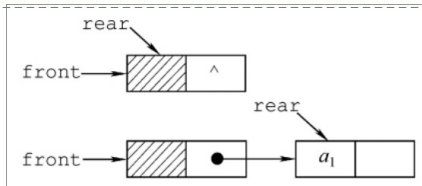


图 3.9　带队头结点的链式队列

(3)双端队列

这里是没有头结点，链栈指针指向栈顶结点。
但是还有的链栈是有头结点的！
YYK
2026/07/25 15:08

这个要读清楚题目，题中明显规定了不存在头结点，所以对于 C 选项来说，把末尾的循环指针指向被删除的表头结点的下一个节点是很困难的一件事情，需要遍历链表
YYK
2026/07/25 15:24

分为带头结点和不带头结点的
它不能根据头指针和尾指针瞬间计算链表长度
链式队列的长度不是不受限制的，还受到空间限制，这是必然的！
YYK
2026/07/26 11:12

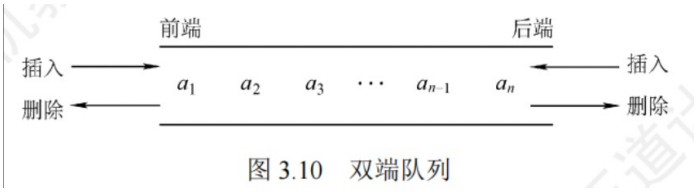


图 3.10　双端队列

（4）循环队列判断队空还是队满，关键在于队头移动和队尾移动的判定，三种方法解决队空和队满相同标志的方法，一定要记牢；关键也在于，写出循环队列中有一个元素的时候，如何表示该循环队列，然后倒推元素为空的时候

(四)多维数组的存储

数组按行存储、按列存储

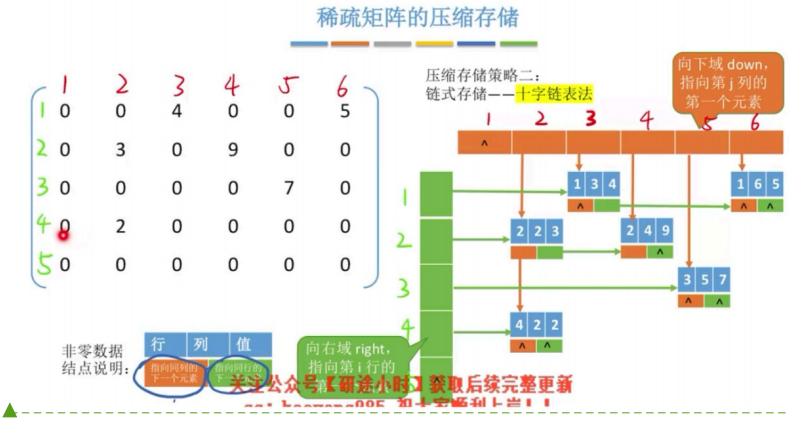
(五)特殊矩阵的压缩存储

①对多个相同值的元素分配一个空间

② 0 不存储

③稀疏矩阵

③稀疏矩阵：三元组(i，j，a_{ij})来存，失去随机存取的特性。三元组表可以用 数组or十字链表 来存。要存**三元组表+行、列数+非零元素个数**



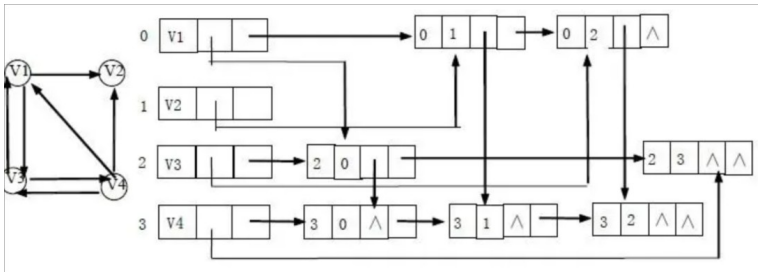
看清楚十字链表法的记录规则，记录了行号和列号，还记录了邻居的关系，方便多种方式来访问矩阵
YYK
2026/11/15 15:36

三元组表：用于存储稀疏矩阵

	i	j	tu
0	6	7	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	6	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

M.data

十字链表（也可以存储稀疏矩阵）



(六)栈、队列和数组的应用

（1）栈：括号匹配、表达式求值、递归

表达式求值：

①超级无敌加括号匹配法，即加完括号把字符移动到括号后面即可。

②遇到操作数直接加入后缀表达式、遇到左括号则压入栈、遇到右括号则弹出左括号内所有的操作符、遇到优先级高的操作符则入栈，若栈内的运算符优先级高于当前未入栈的运算符，那么出栈直到遇到一个优先级低于它的运算符或者遇到左括号，则弹出

或者也可以这么说，**先把栈内的优先级高于等于它的操作符全部依次弹出**，然后再将它放

递归算法效率相对较慢，把递归算法转化为非递归算法，效率是提升的，也不一定仍然需要栈了，例如斐波那契数列，可以用数学公式了，不需要递归操作了
YYK
2026/07/27 11:03

有个最简单的就是画图，画树就好了
基本上算秒杀
中：左根右
先：根左右
后：左右根
SY
2026/07/24 20:40

在加括号的时候也要对正常的A*B/C 进行依次加括号
YYK
2026/07/27 11:21

这个规则是最重要的！
YYK
2026/07/27 10:49

入栈中即可，记得是高于等于它就要弹出，例如栈内存放的是加法，然后又来了一个减法

那么就需要先把加法号弹出，然后把减法号放入栈内

表 3.1　中缀表达式 A+B*(C-D)-E/F 转后缀表达式的过程

步	待处理序列	栈内	后缀表达式	扫描项	说　明
1	A+B*(C-D)-E/F			A	A 加入后缀表达式
2	+B*(C-D)-E/F		A	+	+入栈
3	B*(C-D)-E/F	+	A	B	B 加入后缀表达式
4	*(C-D)-E/F	+	AB	*	*优先级高于栈顶，*入栈
5	(C-D)-E/F	+*	AB	(	(直接入栈
6	C-D)-E/F	+*(	AB	C	C 加入后缀表达式
7	-D)-E/F	+*(	ABC	-	栈顶为(，-直接入栈
8	D)-E/F	+*(-	ABC	D	D 加入后缀表达式
9	)-E/F	+*(-	ABCD	)	遇到)，弹出-，删除(
10	-E/F	+*	ABCD-	-	-优先级低于栈顶，依次弹出*、+，-入栈
11	E/F	-	ABCD-*+	E	E 加入后缀表达式
12	/F	-	ABCD-*+E	/	/优先级高于栈顶，/入栈
13	F	-/	ABCD-*+E	F	F 加入后缀表达式
14		-/	ABCD-*+EF		字符扫描完毕，弹出剩余运算符
			ABCD-*+EF/-		结束

②递归调用

1. 函数参数与返回地址

传入参数：递归函数调用时传入的参数值（如阶乘函数中的 n）。

返回地址：记录调用该函数后应该返回执行的代码位置（即调用点的下一条指令地址）。

2. 局部变量

函数内部定义的变量（如计数器、临时值等）。每次递归调用都会创建独立的副本，避免变量值相互干扰。

3. 上下文保存

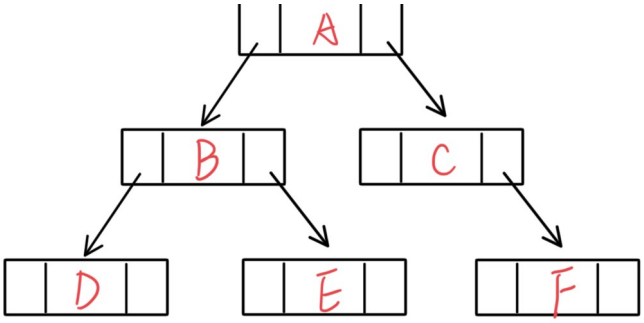
寄存器状态：保存调用前的寄存器值（如程序计数器、基址指针等），以便函数返回后恢复执行环境。



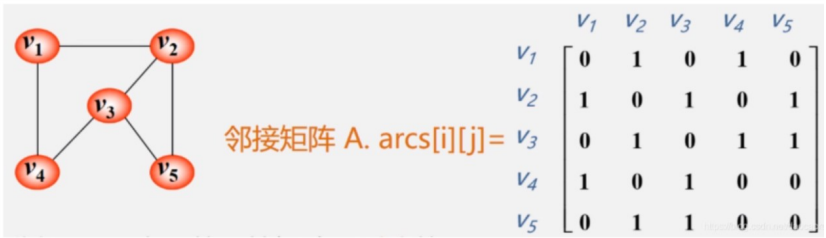
（2）队列：层次遍历、缓冲区（高速设备与低速设备速度不匹配的问题）、多队列（CPU 调度算法）、FIFO 替换算法

进程调度
缓冲区
广度优先搜素遍历
SY
2026/07/24 18:09

二叉链表



邻接矩阵



KMP 算法：

暴力解决时间复杂度为 m*n

但是 KMP 算法的时间复杂度为 n

四、树与二叉树

(一)树的基本概念

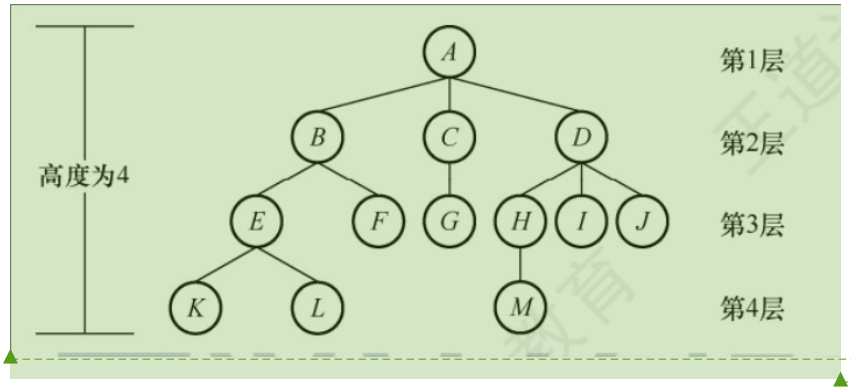
没有节点，叫空树，

n 个节点的树一定只有 n-1 条边，否则就有环存在了

祖先是指从该节点到根节点路上沿途出现的节点。

区分兄弟节点和堂兄弟节点。

子孙既包括孩子也包括孙子、重孙子；



深度和高度的定义不同，导致了树的深度和高度是是一样的，但是节点的深度和高度是不一样的；
YYK
2026/10/10 15:45

注意树的高度的定义
树的高度（ Height ）：从根结点到最远叶子结点的最长路径上的边的数量。
结点的高度：从该结点到其最远叶子结点的最长路径上的边的数量。

YYK
2026/10/10 15:44

树的度=最大的节点的度

分支节点一定不是叶节点，即它一定有孩子

树的路径长度=所有节点到根节点路径长度的总和

要区分树的带权路径和节点的带全路径，这是不同的

YYK
2026/10/10 15:46

树的性质：

非空二叉树性质： n0=n2+1

SY
2026/07/27 15:20

①$N=n_0+n_1+n_2+\ldots+n_n=0*n_0+1*n_1+2*n_2+\ldots+n*n_n+1$
②非空二叉树满足$n_0=n_2+1$
③度为m的树第i层上最多m^{i-1}个结点
④高度为h的满m叉树一共有$\frac{m^h-1}{m-1}$个结点
⑤具有n个结点的完全二叉树的高度为$\lceil \log_2(n+1) \rceil$或$\lfloor \log_2 n \rfloor + 1$
⑥对完全二叉树按照层序进行编号，i=1，2，…，n：
❶i为偶数时，父结点为i/2，i是父结点的左孩子；i为奇数时，父结点为（i-1）/2，i是父结点的右孩子
❷i的左孩子为2i或不存在，i的右孩子为2i+1或不存在
❸i≤$\lfloor n/2 \rfloor$时，i为分支结点；i>$\lfloor n/2 \rfloor$时，i为叶子结点
❹若n为奇数，则每个分支结点都有左右孩子；若n为偶数，则i=n/2（最后一个分支结点）只有左孩子
⑦完全二叉树中度为1的结点（n_1）只可能有1个或者0个
⑧树越高越细（每层结点数量少），越矮越胖（每层结点数量多）
⑨完全二叉树只有最后两层可能出现叶结点

①树的节点数 n=所有节点度数之和+1

07. 设有一棵度为 3 的树，其中度为 3 的结点数 $n_3=2$，度为 2 的结点数 $n_2=1$，叶结点数 $n_0=6$，则该树的结点总数为（　）。

A. 12　　B. 9　　C. 10　　D. ≥9 的任意整数

这也太重要了！！！

YYK
2026/07/29 11:01

该题：
节点数
=2+1+m+6=m+9
度数 =6+2+m=m+8
无法求出具体的 m 值，所以至少有 9 个节点

YYK
2026/07/28 11:45

②度为 m 的树，在其第 i 层，最多有 m^{i-1} 个节点

③高度为 h 的 m 叉树，最多有 $1+m+m^2+m^3$.....

m^i -1

SY
2026/07/27 15:26

④度为 m 的具有 n 个节点的树，最小高度为，自己算吧，把前面几层铺满就行

⑤度为 m，具有 n 个节点的树的最大高度为 n-m+1，因为在树中至少要出现一个度为 m 的节点，否则不符合题意

但这并不意味着至少某一层的节点数就一定等于 m，有可能一定是大于 m 的，例如下图度为 4 的树，不存在某一层节点数一定为 4

YYK
2026/07/28 11:36

(二)二叉树

1.二叉树的定义及其主要特征

度为 2 的树至少有根节点+两个子节点（保护度为 2 的特征），但二叉树不需要这样，即它最少可以为空节点。

度为 2 的树，孩子的相对顺序是根据另一个孩子所确定的，如果没有另一个孩子，也就不需要确定孩子的顺序。但是二叉树的孩子必须确定顺序！

42.【2017 统考真题】要使一棵非空二叉树的先序序列与中序序列相同，其所有非叶结点须

满足的条件是（ ）。

A. 只有左子树　　B. 只有右子树

C. 结点的度均为 1　　D. 结点的度均为 2

自己忘记了节点度均为 1 包含了只有左子树和只有右子树，但是只有左子树显然不对，自己搞错了方向，这时候就已经可以同时排除 AC 了，自己还要去往度为 2 的角度去思考，实在愚蠢！
YYK
2026/07/30 11:40

双亲节点的编号问题：如果编号为 i，那么它的左孩子为 2i，右孩子为 2i+1，另外，它的父亲节点位 i/2 向下取整

①二叉排序树：中序排列的二叉树；左子树小于右子树

②平衡二叉树：树中任意一个节点的左子树和柚子树的高度之差的绝对值不超过 1.

③正则二叉树：每个分支节点都有 2 个孩子

二叉树的性质：

太重要了，多理解多背！
YYK
2026/07/28 14:29

①叶子节点数=度为 2 的节点数+1

下面是针对完全二叉树：

②对完全二叉树编号 1~n,最后一个分支节点的编号为 n/2 向下取整；完全二叉树只可能出现一个度为 1 的节点，且它的编号为最后一个分支节点 n/2 向下取整，在该节点之后的一定是叶子结点；

如果一个节点没有左孩子、必是叶子结点

如果只有一个根节点，那么我们认为这也算是叶子结点，这种特殊情况一定要考虑在内！！！空树、只有根节点的树！
YYK
2026/07/28 14:53

n个结点的完全二叉树树高 h=$\lceil log_2 n \rceil + 1$

⑤ 若 n 为奇数，则每个分支结点都有左、右孩子；若 n 为偶数，则编号最大的分支结点（编号为 $n/2$）只有左孩子，没有右孩子，其余分支结点都有左、右孩子。
⑥ 当 $i > 1$ 时，结点 i 的双亲结点的编号为 $\lfloor i/2 \rfloor$。
⑦ 若结点 i 有左、右孩子，则左孩子编号为 $2i$，右孩子编号为 $2i+1$。
⑧ 结点 i 所在层次（深度）为 $\lfloor \log_2 i \rfloor + 1$。

完全二叉树总共 n 个节点，最后一个分支节点的编号为 N/2 向下取整

12. 已知一棵完全二叉树的第 6 层（设根为第 1 层）有 8 个叶结点，则完全二叉树的结点个数最少是（ ）。

A. 39　　B. 52　　C. 111　　D. 119

思考一下为什么是最少（我之前认为就一种情况啊，何来最字），即自己忽略了一种情况，第六层也可以是倒数第二层，部分分支节点有孩子，部分分支节点没孩子即为叶子结点！这种情况很容易忽略！
YYK
2026/07/29 10:25

16. 一棵有 124 个叶结点的完全二叉树，最多有（ ）个结点。

A. 247　　B. 248　　C. 249　　D. 250

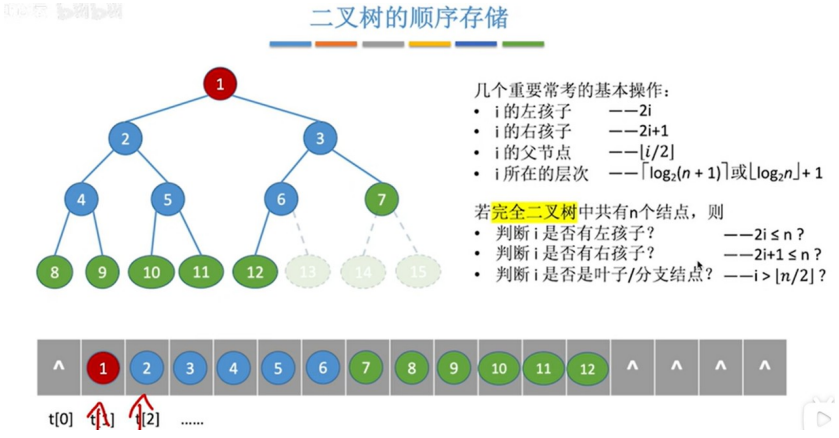
想一下为什么又出现“最”字了，即又有两种情况，在二叉树中，n1 的值要么为 0 要么为 1，则为 1 时肯定是最后一层取代了倒数第二层的第一个叶子节点的位置。 n0=n2+1; 当总节点数为偶数时，一定有 n1=1，因为 n0 和 n2 的和一定为奇数，不可能为偶数的！
YYK
2026/07/29 10:40

2. 二叉树的顺序存储结构和链式存储结构

	双亲表示法	孩子表示法	孩子兄弟表示法
特点	以一组连续的存储单元存储树的结点的双亲	将每个结点的孩子视为以单链表存储的线性表 n个结点又组成另一个基于顺序表的线性表	对树的每个结点，按照“左孩子右兄弟”的原则转换为二叉树 并用二叉链表存储
优点	寻找双亲方便	寻找孩子方便	寻找孩子方便
缺点	寻找孩子需要遍历整个结构	寻找双亲需要遍历整个结构	寻找双亲需要遍历整个结构

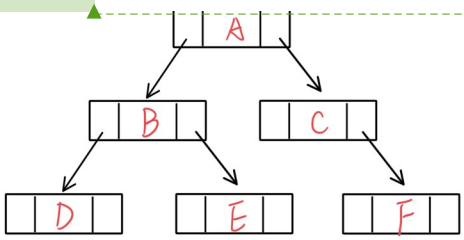
（1）顺序存储结构

编号为 i 的节点放入编号为 i-1 的数组中，数组元素为 0 代表该节点不存在。



（2）链式存储结构

二叉链表（左指针、数据 data、右指针）



n 个节点，必有 n+1 个空链域

YYK
2026/07/28 14:35

三叉链表（左指针、数据 data、右指针、父指针）

3.二叉树的遍历

NLR、LNR、LRN，其中 N 代表根节点，L 代表左、R 代表右边

删除树时，要先删除左孩子和右孩子，再删除自身节点，这与后序遍历的顺序一模一样

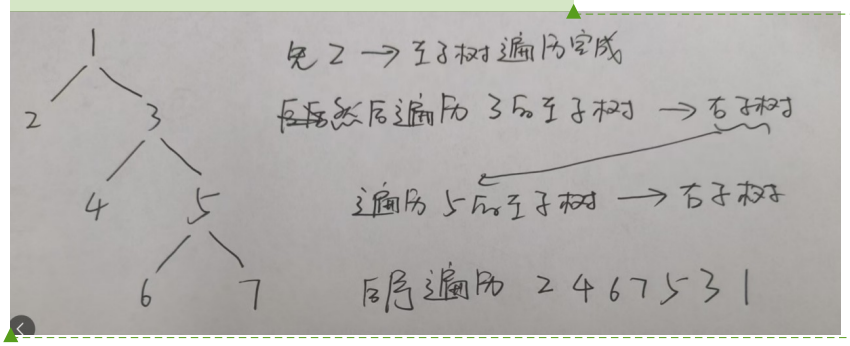
YYK
2026/07/30 10:03

层序遍历需要借助队列，出队后才能把它的孩子们放入队列中！

中序遍历+任意一个遍历的序列=唯一二叉树！

太重要了！

YYK
2026/07/29 11:12



自己犯了一个非常愚蠢的错误，连后序遍历的都不会了

YYK
2026/07/30 10:46

4.线索二叉树的基本概念和构造

线索二叉树是一种存储的非线性树形链表，属于物理结构+逻辑结构！

YYK
2026/07/30 10:22

（1）利用二叉链表的空指针域来存放其前驱或者后继的指针，加快查找节点前驱和后继的速度

目的就是为了快速找到前驱和后继节点，普通的二叉树结构只能反映出父子结构，不能反应前驱后继关系

YYK
2026/07/29 11:51

线索化的本质就是对树进行依次遍历，只有遍历过后存储完相关信息的才叫线索二叉树

YYK
2026/07/29 11:52

lchild	ltag	data	rtag	rchild

图 5.16　线索二叉树的结点结构

域的含义如下：

$$ltag=\begin{cases}0, & lchild域指示结点的左孩子\\ 1, & lchild域指示结点的前驱\end{cases}$$

$$rtag=\begin{cases}0, & rchild域指示结点的右孩子\\ 1, & rchild域指示结点的后继\end{cases}$$

太重要的表格了，这种结构叫做线索链表！

YYK
2026/07/29 11:41

比较奇特的一种结构，实现了双向线索链表，构建一个闭环的遍历结构，从前往后，从后往前都比较方便。

从头结点到达根节点后，必须继续向左下递归才能找到真正的第一个访问节点。这是实现中序线索遍历不可或缺的一步！

YYK
2026/07/29 11:44

（2）带头结点的线索二叉树

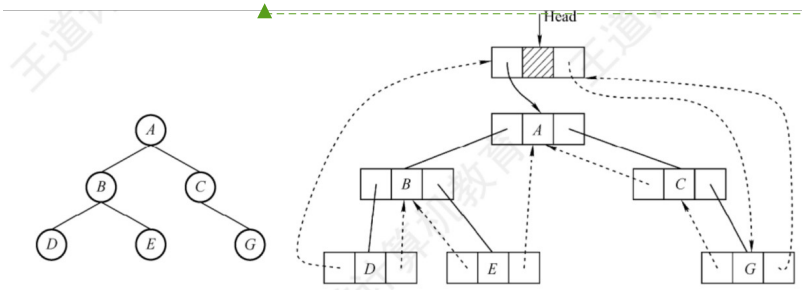


图 5.18　带头结点的中序线索二叉树

关于头结点：
左孩子指向根节点，是为了保证有对树访问的入口。另外实现一些特殊操作例如插入删除，代码量也会减少

YYK
2026/07/29 14:16

目的还是统一化操作，让最后一个节点的“虚拟”后继为头结点。
①同时也方便了反向遍历整棵树。
②通过最后一个节点 ->rchild == 头结点 可以直接判断：“这是序列的最后一个节点”
③从头结点可以迅速找到最后一个节点

YYK
2026/07/29 14:14

对于头结点的描述：

左指针 tag=0，指向根节点

右指针 tag=1，线索化指向最后一个节点

遍历的第一个节点的左线索指向头结点

遍历的最后一个节点的右线索指向头结点

指向头结点后
①通过 第一个节点 ->lchild == 头结点 就能直接判断：“这是序列的第一个节点”（因为只有它的前驱是头结点），此时可以结束循环遍历

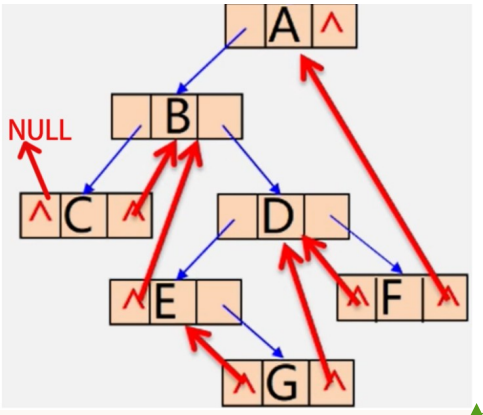
YYK
2026/07/29 14:17

①序列形成闭环：从中序序列的第一个结点向后遍历，最终通过最后一个结点的线索回到头结点。

⑤统一遍历逻辑：无论正向或逆向遍历，均以头结点为起点和终点，简化代码实现。

YYK
2026/07/29 14:22

（3）不带头结点的中序线索二叉树



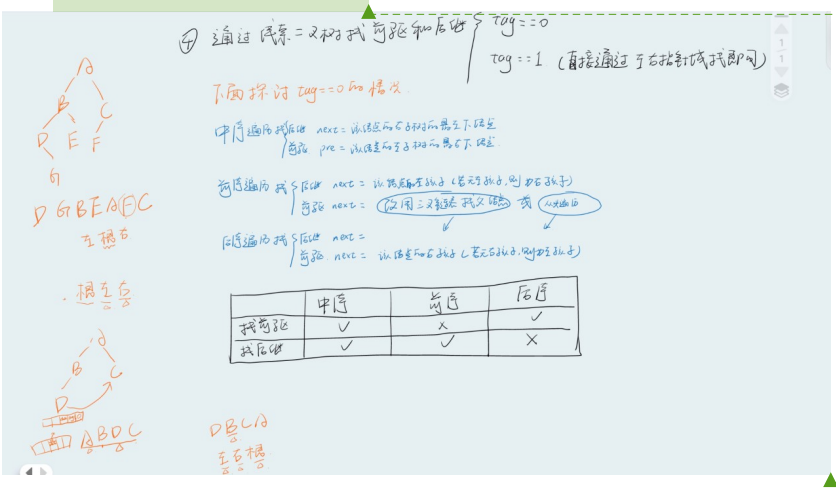
在实际应用中，最广泛使用的是 **中序线索二叉树**。因为通过中序线索，可以用如下方式访问整棵树：

1. 从最左下角的结点开始
2. 通过右线索一步步访问中序后继
3. 直到结束，不需要递归或辅助栈

这种遍历方法最简单、最直接。

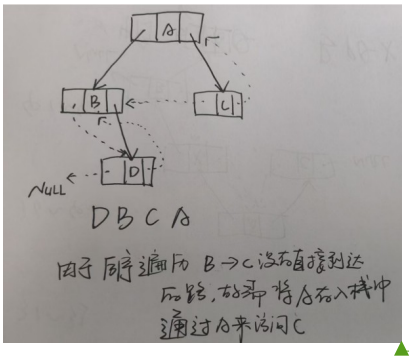
整个中序遍历的结果是 CBEGDFA ， C 没有前驱，那么就把他的左线索设置为 NULL 即可， A 也没有直接后继，也把他的右线索设置为 NULL
YYK
2026/07/29 14:05

（4）通过线索二叉树找前驱和后继节点



在后续线索二叉树上找后继时需要知道结点双亲，需要采用带标志域的三叉链表作为存储结构
YYK
2026/07/29 14:34

YYK
2026/07/30 10:12



后序线索树的遍历需要栈的支持！

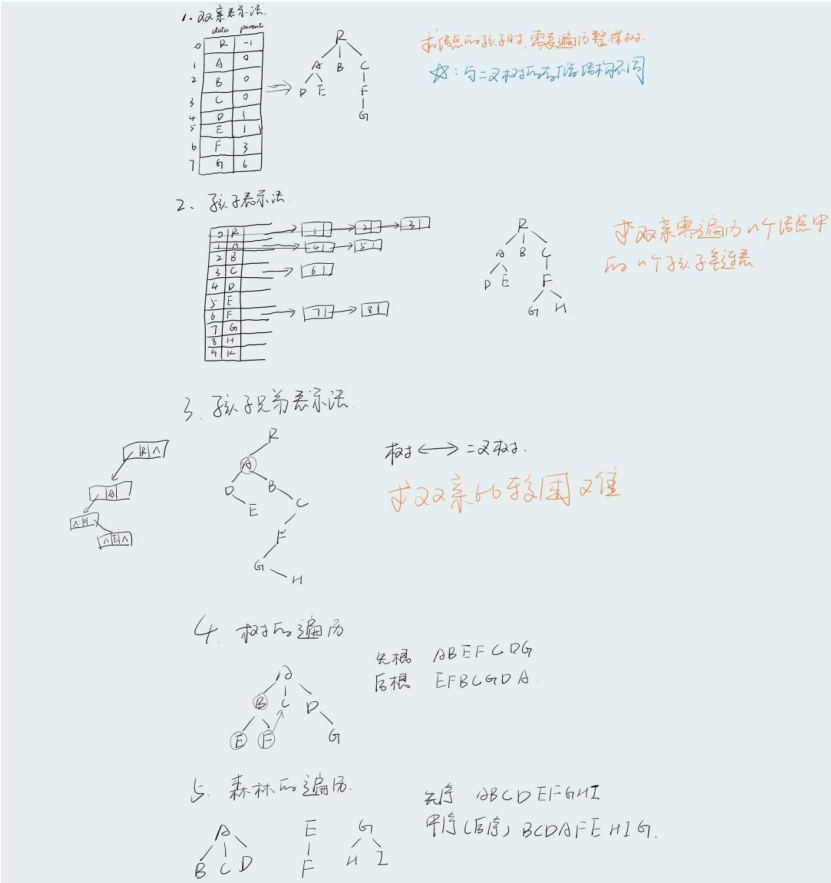
YYK
2026/07/30 10:55

(三)树、森林

1.树的存储结构

顺序存储结构：数组
链式存储结构：线性链表

YYK
2026/07/30 11:44



15. 【2009 统考真题】将森林转换为对应的二叉树，若在二叉树中，结点 u 是结点 v 的父结点的父结点，则在原来的森林中，u 和 v 可能具有的关系是（ ）。

I. 父子关系　II. 兄弟关系　III. u 的父结点与 v 的父结点是兄弟关系

A. 只有 II　　B. I 和 II　　C. I 和 III　　D. I、II 和 III

16. 【2011 统考真题】已知一棵有 2011 个结点的树，其叶结点个数为 116，该树对应的二叉树中无右孩子的结点个数是（ ）。

A. 115　　B. 116　　C. 1895　　D. 1896

孩子表示法中，R 指向的 123 都是 R 的孩子！不要搞混

YYK
2026/07/30 11:51

BD 一定要细致
第十六题，一定要分清楚在二叉树中谁是谁的儿子、谁是谁的兄弟

YYK
2026/07/30 15:04

2.森林与二叉树的转换

3.树和森林的遍历

森林的遍历也是一棵树一棵树依次遍历的

YYK
2026/07/30 14:48

表 5.1　树和森林的遍历与二叉树遍历的对应关系

树	森　林	二　叉　树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

树是没有中序遍历的，因为他的孩子可能多于 2 ，那么也就无从谈起左根右了！
YYK
2026/07/30 11:56

(四)树与二叉树的应用

1.哈夫曼(Huffman)树和哈夫曼编码

即使，原树的真实节点数为 n-1,但是构造哈夫曼树之后结点数是要增加的，为 2n-1。

前缀编码是指没有一个编码是另一个编码的前缀，否则就乱套了。

即哈夫曼树的度为 2 的节点为 n-1 ，度为 0 的节点为 n ，满足二叉树 n0=n2+1 ；并且哈夫曼树没有度为 0 的节点
YYK
2026/07/31 11:03

但是， A0,B10,C110,D111 ，虽然 B 为 10 时 C110 的后缀，但是咱们强调的是前缀不能相同！
2014 年考了前缀编码的判别方法，一定要关注！定义！
YYK
2026/07/31 10:35

用一组权值构造出的哈夫曼树可能是不同的，但是 WPL 一定是相同的！

哈夫曼树的带权路径长度=所有叶节点的带权路径长度或者所有分支节点的权值之和

这是有两种计算方法的，请注意，一定不要搞混！，两个方法都可以，本质还是对哈夫曼树定义的延伸！
YYK
2026/07/31 11:36

加权平均长度=（各个权值*对应长度）/权值之和

哈夫曼编码长度+1=哈夫曼树的最大高度

注意与带权路径之和 WPL 的区别，这个在 2023 年考了真题！
YYK
2026/07/31 12:13

数据结构-哈夫曼树和并查集_哔哩哔哩_bilibili

2.并查集及其应用

双亲表示法

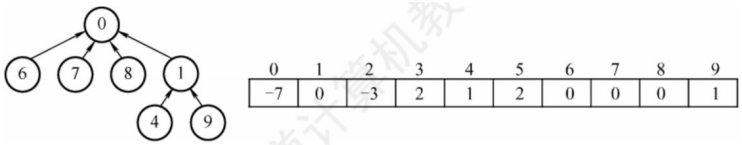


图 5.30　$S_1 \cup S_2$ 可能的表示方法

用一个数组表示双亲表示法，数组中的内容指向的是它的双亲
YYK
2026/07/31 14:23

路向北找爸爸、合并两个集合（合并时是把第二个集合的根节点放到第一个集合的根节点的下面，当做它的儿子！）

并查集中最核心的两个操作是：①查找，查找两个元素是否属于同一个集合；②合并，若两个元素不属于同一个集合，且所在的两个集合互不相交，则合并这两个集合。假设初始长度为 10（0～9）的并查集，按 1-2、3-4、5-6、7-8、8-9、1-8、0-5、1-9 的顺序进行查找和合并操作，最终并查集共有（ ）个集合。

A. 1　　B. 2　　C. 3　　D. 4

例如下题，78 为一个集合，7 为根节点，此时要合并 89，则 9 要放在根节点 7 的下面，而不是放在 8 的下面！
YYK
2026/07/31 11:41

3.堆及其应用

五、图

(一)图的基本概念

顶点数 V 和边数 E，图不能为空，即最少要有一个顶点！

①有向图

$$G_1 = (V_1, E_1)$$
$$V_1 = \{1, 2, 3\}$$
$$E_1 = \{<1, 2>, <2, 1>, <2, 3>\}$$

②无向图

$$G_2 = (V_2, E_2)$$
$$V_2 = \{1, 2, 3, 4\}$$
$$E_2 = \{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)\}$$

王道计算机

③简单图与多重图

不存在重复的边、不存在自身到自身的边为简单图

否则为多重图

④度、入度、出度

有向图的某个顶点 v 的度=入度+出度，所有顶点的入度=出度

无向图的度=与顶点相连的边数

如果有 n 个顶点，那么其中顶点的度最大可达 2n-2 ，因为可以加双向的边
YYK
2026/08/01 14:49

⑤ n 个顶点，若有大于 n-1 的边数，则一定有环存在

⑥简单路径、简单回路都是指，没有重复的顶点存在

⑦两个顶点之间没有边存在，那么距离就设置为∞

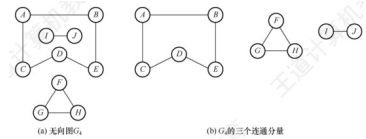
⑧子图：就是图的一部分

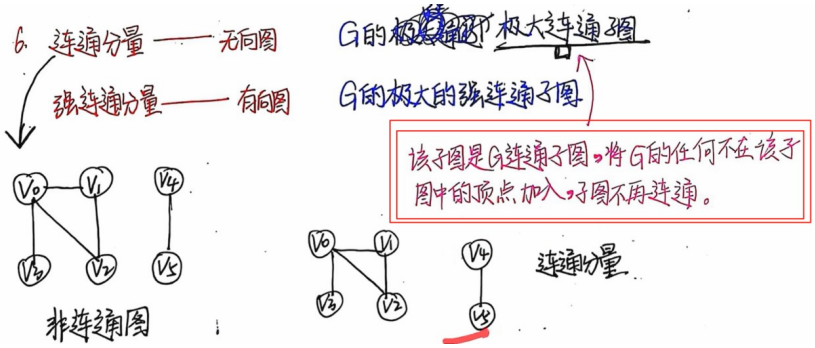
生成子图：包含了所有顶点，但是没有包含所有边

⑨连通、连通图、连通分量

在无向图中，任意两个顶点都是连通的，才叫连通图

若是非连通图，则叫连通分量





⑩强连通图、非强连通图、强连通分量

强连通图：在有向图中任意两个顶点之间都有路径存在

假设有 n 个顶点，则至少要有 n 条边才能构成强连通图（这里即一个环）

这里要注意，如果给你一些子顶点、一些子边，你要保证这些子顶点和边是配套的（可能考概念题），否则是构不成图的。
YYK
2026/08/01 14:45

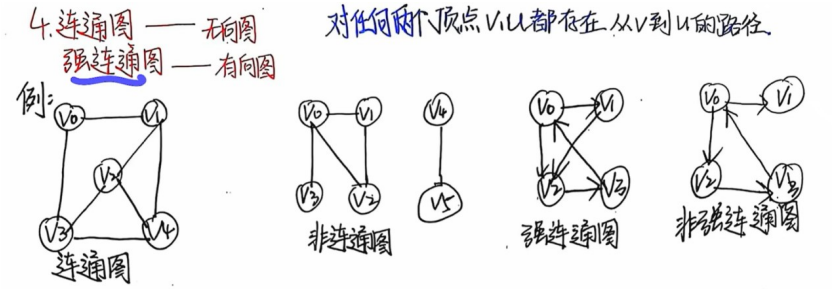
如果一个一个图是连通的，那么它本身就是一个连通分量，且仅有这一个连通分量。

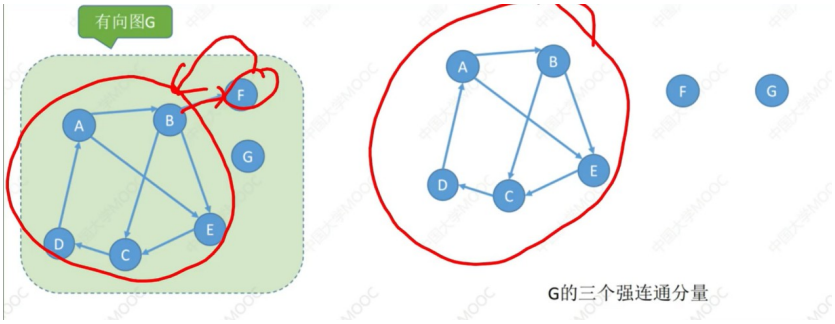
如果一个图是不连通的，那么各个连通的子图，都是该图的连通分量。

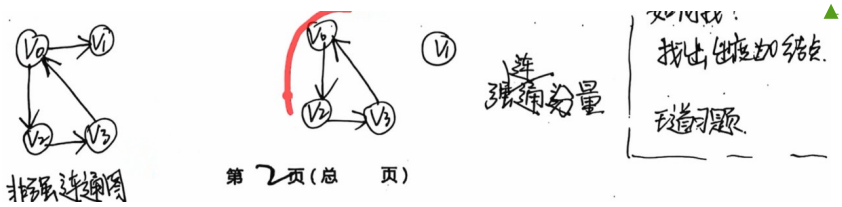
注意在划分连通分量的过程中是不能少边的，即所有的连通分量之和就是原图，例如一个环图 G，它的生成树 G'，显然 G' 就不是一个连通分量，因为它缺失了一条边，不满足连通分量的定义（from 王怡童）
YYK
2026/08/01 15:03

即任意两个顶点之间都有“路径”存在，而不是任意两个顶点之间都直接相连

假设 n 个顶点，若边数小于 n-1，则一定是非连通图
YYK
2026/07/31 14:56







有向图中，F 没有到达其他顶点的路径，所以它需要单独拿出来，当做一个强连通分量，一定要理解强连通图的定义，与无向图的连通图的定义是不一样的

强连通图是所有顶点都要有达到任意顶点的路线

YYK
2026/08/01 14:52

11. 生成树、生成森林

对于连通的无向图来说，生成树就是有 n 个顶点、n-1 条边，构成一个树

对于非连通的无向图来说，每个连通分量构成的树，合起来就是一个森林

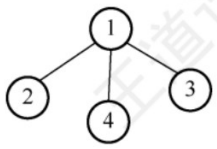


图 6.4　图 G_2 的一个生成树

这里指的是包含所有顶点的极小连通子图（要求保持子图连通并且边数最少的子图，**生成树就是极小连通子图**）

很可能考极大连通子图和极小连通子图（又叫生成树）的区别

极大连通子图（又叫连通分量）要求子图必须连通，尽可能包含可能多的顶点和边

YYK
2026/08/01 09:51

12. 边的权值、网、带权路径长度

带有权值的图才叫网

所有边上的权值之和成为带权路径长度

13. 完全图

无向图：所有顶点之间都有边

有向图：所有顶点之间都有双向的边

14. 稠密、稀疏图

主要针对的是边数，概念比较模糊，一般认定

15. 有向树

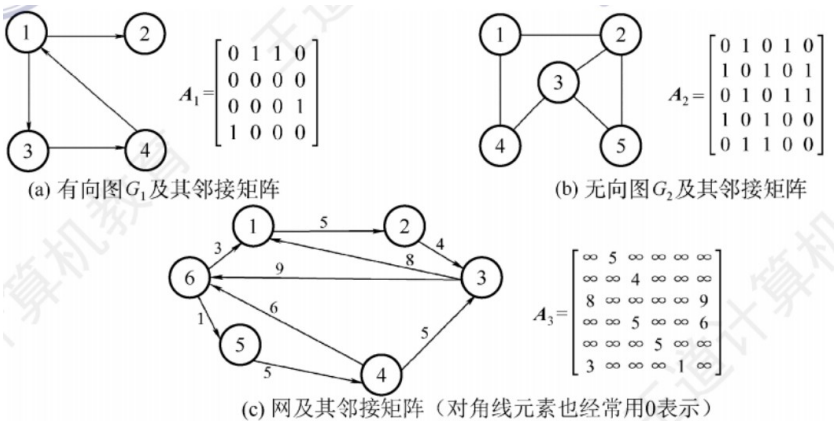
针对于有向图，构造成树那样

常见考点：
对于n个顶点的无向图G，
- 所有顶点的度之和=2|E|
- 若G是连通图，则最少有 n-1 条边（树），若 |E|>n-1，则一定有回路
- 若G是非连通图，则最多可能有 C_{n-1}^2 条边
- 无向完全图共有 C_n^2 条边

对于n个顶点的有向图G，
- 所有顶点的出度之和=入度之和=|E|
- 所有顶点的度之和=2|E|
- 若G是强连通图，则最少有 n 条边（形成回路）
- 有向完全图共有 $2C_n^2$ 条边

(二)图的存储及基本操作

1.邻接矩阵



在邻接矩阵中为 0 或者∞的都代表这条边不存在，而不是这条边存在，权重为 0

但是在 2011 年真题当中，只在不存在的边设置为∞，对角线（自身到自身）设置为 0，这个答案一直沿用

YYK
2026/08/02 09:08

最简单的一种情况，只需要关注如果没有边的话记为 0 或者无穷大即可。

该矩阵存储的空间复杂度为 O（n^2）

第 i 行或者第 i 列的元素之和正好是无向图某个顶点的度

要判断无向图中共有多少条边，需要对整个邻接矩阵进行遍历扫描，所以时间复杂度很高

$A^2[i][j]$代表从 i 顶点到 j 顶点长度为 n 的路径的数目（2015 真题）

2.邻接表

在邻接矩阵的方法上，解决了稀疏矩阵的存储的浪费空间的问题

YYK
2026/08/02 09:13

关键点在于为每个顶点建立一个单链表，依次链接与其相连的的顶点。

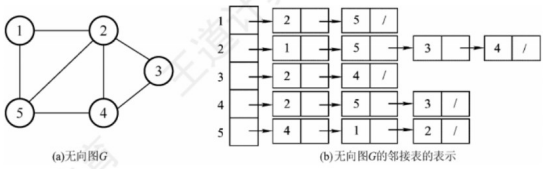
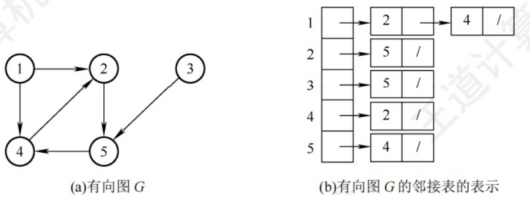


图 6.7　无向图邻接表表示法实例



再次注意观察有向图的邻接表中边表节点的特点，值存储了头结点。只存储边的头结点！！！

YYK
2026/08/02 11:01

注意对于顶点和邻接的顶点的数据结构是不一样的

要学会记忆这个链表的结构

YYK
2026/10/16 14:29

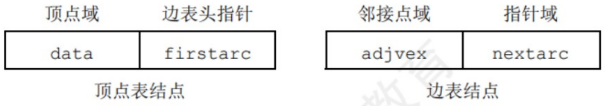


图 6.6　顶点表和边表结点结构

构件邻接表的空间复杂度为无向图 O（V+2E）,有向图 O（V+E）

非常重要的知识点：无向图的边在邻接表存储过程中存储了两次

YYK
2026/08/02 10:58

（2）邻接表法和邻接矩阵法的差异

①邻接表中可以很快找出所有顶点的邻边，邻接矩阵中需要完全读取一行

②邻接表中求有向图的入度需要访问整个邻接表

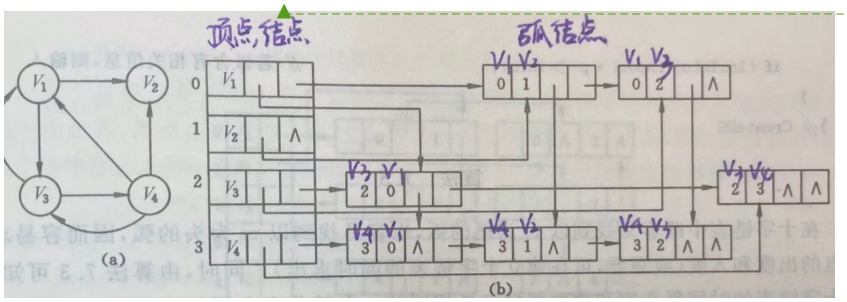
③图的邻接表并不唯一，因为各个顶点的链接顶点的顺序可以更改的

4.邻接多重表、十字链表

十字链表（有向图）：

在十字链表中，非常容易找到与顶点相连的入度和出度的边，但图的十字链表不是唯一的（边的顺序问题），但十字链表映射的图是唯一的

YYK
2026/08/02 10:08



画十字链表的方法：

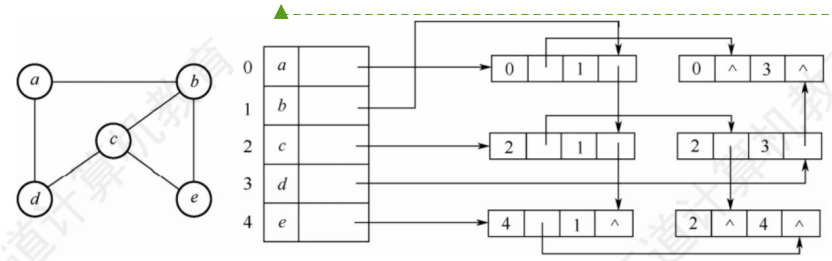
①先画出所有顶点，每个顶点有三个数据元素：元素顶点号+入度指针+出度指针

②在每个顶点后面画出所有与之链接的单向出度的边，每个边有四个数据元素：出度的尾节点+入度的头结点+头结点相同的下一个边指针+尾节点相同的下一个指针；

邻接多重表（无向图）：

每个边只在邻接多重表中出现一次！这个非常重要

YYK
2026/08/02 10:48



画邻接多重表的方法：

①先画出所有顶点，每个顶点有两个数据元素：元素顶点号+指针（该指针指向与边结点中相同的节点元素后面的那个指针区域）

②再画图中所有的边（不重复的画），每个边有四个数据元素：头顶点编号+与头顶点相同的下一条边的指针+尾顶点编号+与尾顶点相同的下一条边的指针+info 权重（可有可无）

	邻接矩阵	邻接表	十字链表	邻接多重表
空间复杂度	O(|V|²)	无向图 O(|V| + 2|E|) 有向图O(|V| + |E|)	O(|V| + |E|)	O(|V| + |E|)
找相邻边	遍历对应行或列 时间复杂度为O(|V|)	找有向图的入边必须遍历整个邻接表	很方便	很方便
删除边或顶点	删除边很方便，删除顶点需要大量移动数据	无向图中删除边或顶点都不方便	很方便	很方便
适用于	稠密图	稀疏图和其他	只能存有向图	只能存无向图
表示方式	唯一	不唯一	不唯一	不唯一



(三)图的遍历

从图中的某一个顶点出发，按照某种搜索方式访问图中所有顶点，且每个顶点仅被访问一次（所以设计了一个 visitied[i] 数组来存数该结点是否被访问过）。否则就会产生循环遍历，失去了遍历的意义

YYK
2026/08/01 14:46

1.广度优先搜索

（1）基本概念

①类似于树的层序遍历

②借助队列存储临时的节点，当某个节点出队时，需要把它的链接的节点扫描入队

③非递归算法、不需要回退操作

④设置一个 visited 数组来设置节点状态

（2）性能分析

队列最长存储 V-1 个节点，即其余 v-1 个结点全都与剩余的那个节点相连，故空间复杂

度为 O(V)

采用邻接表时，每个顶点和每条边都需要访问一次，所以时间复杂度是 O（V+E），

空间复杂度为 O（n）；采用邻接矩阵存储时，查找每个顶点的邻接点时间为 O

（V），所以时间复杂度为 O（V^2）

只需要借助队列即可

YYK
2026/08/05 09:49

BFS 从顶点 0 开始的过程

1. **初始化**：标记所有顶点未访问（数组操作，O(|V|) = O(4)）。
2. 0 入队，标记已访问。
3. **循环直到队列空**：
   - 0 出队，访问它的邻居 [1, 2]（检查边 0→1, 0→2）
     1 未访问 → 入队，标记
     2 未访问 → 入队，标记
   - 1 出队，访问它的邻居 [0, 2]
     0 已访问 → 跳过
     2 已访问 → 跳过
   - 2 出队，访问它的邻居 [0, 1, 3]
     0 已访问 → 跳过
     1 已访问 → 跳过
     3 未访问 → 入队，标记
   - 3 出队，访问它的邻居 [2]
     2 已访问 → 跳过

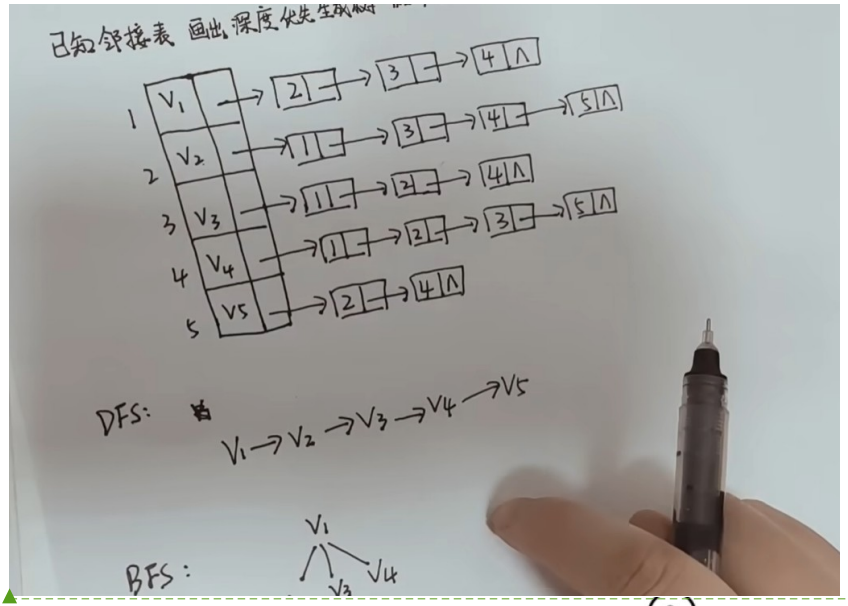
（3）广度优先算法求最短路径

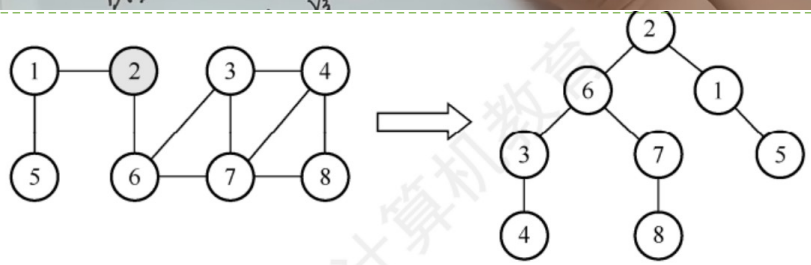
注意广度优先搜索插入队列的过程中，需要按照邻接表中的边的顺序插入到队列中

（4）广度优先生成树

邻接矩阵的广度优先生成树是唯一的，但邻接表（表示法不唯一）的广度优先生成树不是唯一的

是由于邻接表的表示方法不唯一导致了广度优先生成树不唯一，如果已经确认了邻接表的存储顺序，那么广度优先生成树就是唯一的
YYK
2026/10/16 15:11





这个图讲解了如何通过邻接表快速找到广度优先生成树和深度优先生成树

深度：沿着 V1 开始遍历边，逐后加入式子中，然后，遍历下一个 V2 的边，找到未被遍历的节点加入式子；

广度，从第一个节点开始遍历边，加入该点的下一层式子中，。，，，，，
YYK
2026/10/16 15:12

2.深度优先搜索

（1）基本概念

①类似于树的先序遍历

②借助栈存储临时的节点，当某个节点没有可访问的其他邻边时，回退到上一个节点，访问上一个节点的其他邻边

③递归算法、需要回退操作

④设置一个 visited 数组来设置节点状态

⑤若采用邻接矩阵则 BFS 与 DFS 一致，若采用邻接表法则 DFS 和 BFS 不一致

（2）性能分析

空间复杂度为 O(V)

采用邻接表时，每个顶点和每条边都需要访问一次，所以时间复杂度是 O（V+E），

空间复杂度为 O（n）；采用邻接矩阵存储时，查找每个顶点的邻接点时间为 O

（V），所以时间复杂度为 O（V^2）

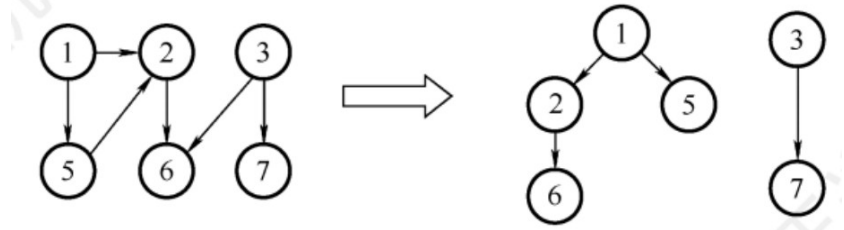
借助栈，最多存储 n 个顶点的空间

YYK
2026/08/05 09:49

（5）深度优先生成树和生成森林

邻接矩阵的广度优先生成树是唯一的，但邻接表（表示法不唯一）的广度优先生成树

不是唯一的



调用几次 DFS ，就会有几个连通分量
任何非强连通图，都需要调用两次及其以上的的 BFS 吗？不一定！例如下图，只需要一次即可实现 BFS

YYK
2026/08/02 11:41

图的遍历和图的连通性：

在图的 DFS 和 BFS 中，添加了第二个 for 循环来选取初始点继续遍历，防止一

次无法遍历图的所有顶点。

(四)图的基本应用

1.最小(代价)生成树

生成树（减去一条边变成非连通图、加上一条边有回路）

（1）基本概念

①若各边的权重不同，则 G 的最小生成树一定唯一存在

反过来可不一定，若权重相同，最小生成树可能唯一存在

YYK
2026/08/05 10:02

②最小生成树的边数=顶点数 n 减去 1

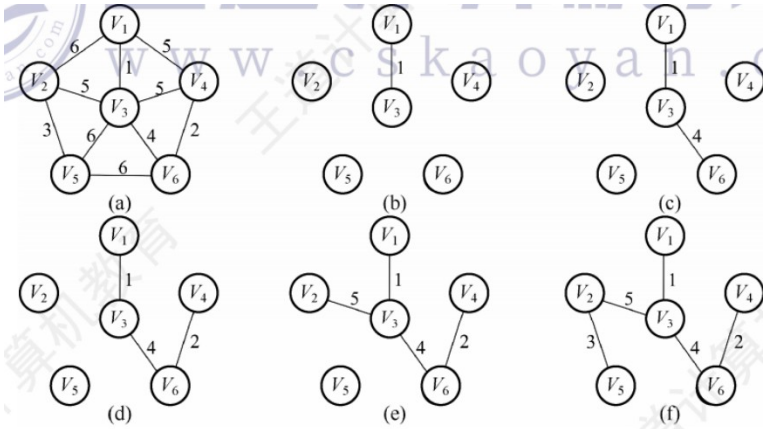
③最小生成树只保证了整个图的代价最低，但并不能保证原图中任意两个节点之间距离最小

④最小生成树是基于贪心算法！

（2）Prim 普利姆算法（选择与上一步最近的一个顶点Point---2P 组合）

这样保证每一次执行之后，这个临时图一定是连通的

YYK
2026/08/02 14:44



使用邻接矩阵的方式时间复杂度为 O（V^2），并不依赖于边的数目，所以适用于边稠密的图的最小生成树

每次都要遍历一整个

YYK
2026/10/16 15:19

算法思路：

首先，初始化空树、然后加入任意一个顶点；在其余所有顶点当中，找到与之树距离最近的顶点，加入树中，循环操作

4. 时间复杂度分析

- **外层循环**：执行 V 次（每次加入一个顶点）。
- **第 1 步（找最小 key）**：需要遍历所有 V 个顶点检查 `inMST` 并比较 key，$O(V)$。
- **第 3 步（更新邻居）**：需要遍历所有 V 个顶点检查边并更新 key，$O(V)$。

所以每轮时间：

$$O(V) + O(V) = O(V)$$

总时间：

$$V \times O(V) = O(V^2)$$

我觉得比较难以理解的在于，整个顶点遍历了一整次 V ，然后每次加入一个顶点之后都要修改全部的 KEY[] 数组和 isMST[] 数组，每次 O（ V ），所以总的时间复杂度为 V^2
YYK
2026/10/18 14:33

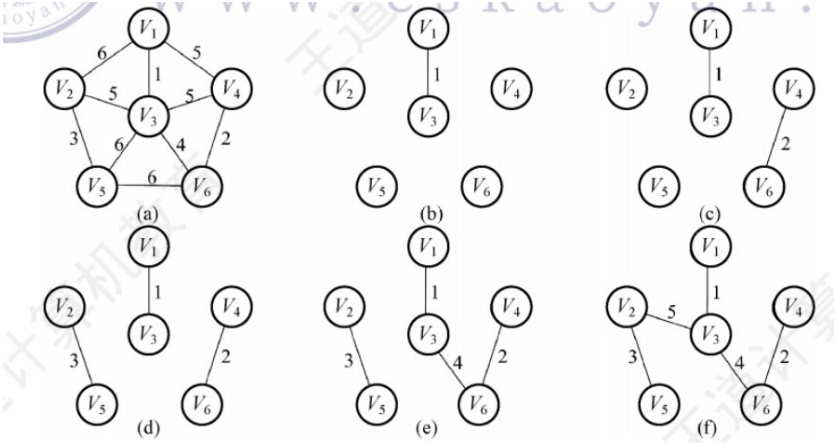
重复 V 次（每次加入一个顶点）：

1. **找最小 key 的顶点**：
   遍历所有 **不在 MST 中** 的顶点，找到 `key[u]` 最小的顶点 u。
   - 第一次时，`key[0]` 最小，所以选顶点 0。
2. **将 u 加入 MST**：
   `inMST[u] = true`。
3. **更新邻居的 key**：
   遍历所有顶点 v：
   - 如果 `inMST[v] == false` 且 `graph[u][v] < key[v]`，则 `key[v] = graph[u][v]`（因为通过 u 到 MST 的边更短）。

（3）Kruskal 克鲁斯卡尔算法（选择原图中权重最小的边）

即使与临时图不连通，也是合理的
YYK
2026/08/02 14:54

最终的终止结果是：没有形成回路、且又 n-1 条边

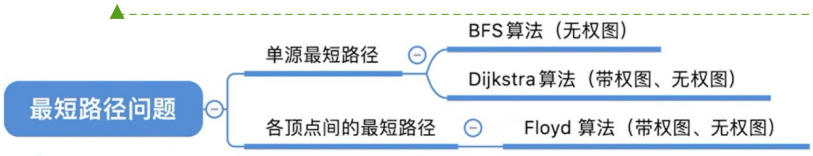


最坏的情况是需要对所有的边都扫描一次 O（E），每次扫描需要 O（$\log_2 E$）的时间，所以时间复杂度为为 O（E $\log_2 E$），所以适用于边稀疏而顶点较多的图

这整个阶段都是排序的阶段，后续扫描获取最小生成树的阶段可以单开一个循环
YYK
2026/10/18 14:45

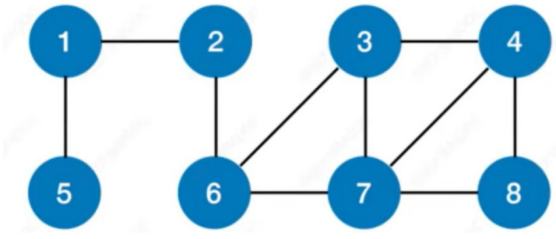
2.最短路径

下面的算法求的是某一个顶点，到其他顶点的最短路径长度，而不是所有顶点之间的路径长度
YYK
2026/08/05 10:46



（1）BFS 求单源最短路径（无权图）

从 2 出发



	1	2	3	4	5	6	7	8
d[ ]	∞	∞	∞	∞	∞	∞	∞	∞
path[ ]	-1	-1	-1	-1	-1	-1	-1	-1

注意这里的 Path 是指上一个顶点的编号

YYK
2026/10/18 14:49

	1	2	3	4	5	6	7	8
d[ ]	1	0	2	3	2	1	2	3
path[ ]	2	-1	6	3	1	2	6	7

```
//求顶点 u 到其他顶点的最短路径
void BFS_MIN_Distance(Graph G,int u){
    //d[i]表示从u到i结点的最短路径
    for(i=0;i<G.vexnum;++i){
        d[i]=∞;       //初始化路径长度
        path[i]=-1; //最短路径从哪个顶点过来
    }
    d[u]=0;
    visited[u]=TRUE;
    EnQueue(Q,u);
    while(!isEmpty(Q)){              //BFS算法主过程
        DeQueue(Q,u);                //队头元素u出队
        for(w=FirstNeighbor(G,u);w>=0;w=NextNeighbor(G,u,w))
            if(!visited[w]){         //w为u的尚未访问的邻接顶点
                d[w]=d[u]+1;         //路径长度加1
                path[w]=u;           //最短路径应从u到w
                visited[w]=TRUE;     //设已访问标记
                EnQueue(Q,w);        //顶点w入队
            }//if
    }//while
}
```

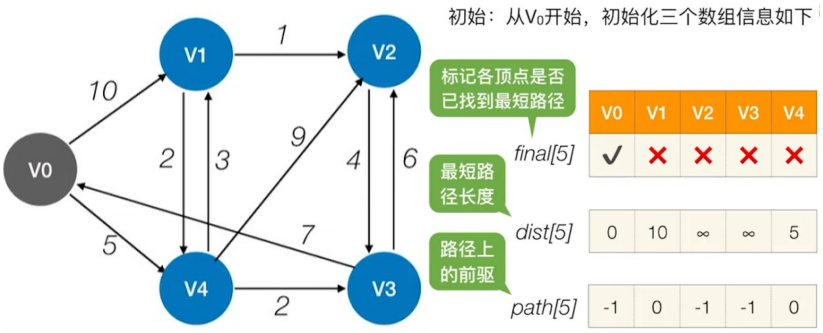
（2）Dijkstra 迪杰斯特拉算法（无权图、带权图）

基于贪心策略
不适合用于带负值的

若要求每对顶点之间的最短路径，那么需要调用 n 次迪杰斯特拉算法！
YYK
2026/08/03 10:55

算法逻辑如下：

①首先进行初始化工作：从初始点到相邻节点的路径信息，并更新三个数组



②开始进行第一轮扫描，在 final 数组中找到未被设置为 ture 的数组，然后找到其中 dist 最小的那个节点，并将其设置为 true；从该节点出发，更新其他节点的数组信息（如果从该节点出发有更短的路径和，则更新 dist）

③按照②的步骤依次进行多轮扫描

性能分析：

使用邻接矩阵、带权的邻接表表示则时间复杂度为 O（V^2）

关键是要对 n 个顶点进行 n 轮扫描

YYK
2026/08/05 10:11

（3）Floyd 弗洛伊德算法（无权图、带权图、带负权图）

基于动态规划
不能解决带有负权回路的问题，这种图可能没有最短路径

在经过每一轮的更新之后，新一轮的路径集合可能不是上一轮路径集合的子集！
YYK
2026/08/03 10:55

算法逻辑如下：

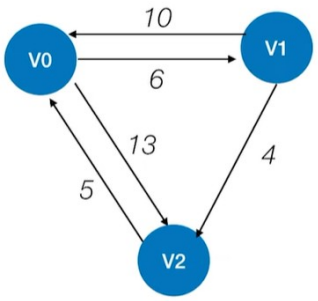
①首先准备一个邻接矩阵和一个基础的 path 矩阵

A=

	V0	V1	V2
V0	0	6	13
V1	10	0	4
V2	5	∞	0

path=

	V0	V1	V2
V0	-1	-1	-1
V1	-1	-1	-1
V2	-1	-1	-1



②然后进行第一轮扫描，允许经过 V0 节点来进行中转，则比较经过 VO 中转之后的路径长度与目前的 A 的数值，若小于则更新 A 数组，同时更新 path 数组

③按照②的步骤执行 n 轮



性能分析：

时间复杂度是 O（V^3），空间复杂度为 O（V^2）

	BFS 算法	Dijkstra 算法	Floyd 算法
无权图	✓	✓	✓
带权图	✗	✓	✓
带负权值的图	✗	✗	✓
带负权回路的图	✗	✗	✗
时间复杂度	O(|V|²)或O(|V|+|E|)	O(|V|²)	O(|V|³)
通常用于	求无权图的单源最短路径	求带权图的单源最短路径	求带权图中各顶点间的最短路径

3.拓扑排序

AOV（有向无环图、用顶点表示活动的网）

算法关键点：找到入度为 0 的节点

拓扑排序可以判断图中是否有环路存在，实现原理就是在查找出度为 0 的节点是，是否存在，若不存在，但是节点还没有全部输出，这就可以判定出现了回路

同时还可以使用深度优先遍历算法判断回路，若在执行发现其邻接节点已经被访问过，并且该邻接节点不是当前节点的父节点（刚刚从这个节点遍历过来）

同时还可以使用求关键路径，如果有环的话，会导致关键路径长度无穷大

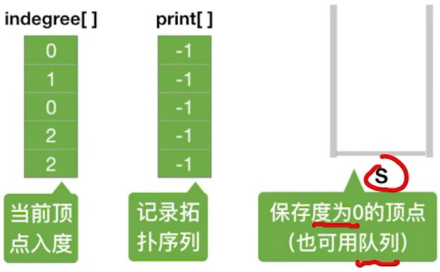
YYK
2026/08/05 09:38

本质上就是找到一个顶点序列，按照这个序列执行该整体的活动，可以顺利执行

YYK
2026/08/04 10:55

每个 AOV 网都有一个或者多个拓扑排序

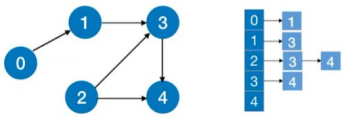
YYK
2026/08/04 10:56



拓扑排序算法的关键在于设置了一个 indegree 数组、设置了一个 print 数组、设置了一个栈或者队列。

Indegree 数组里面保存的是当前所有顶点的入度、print 指的是打印出的序列、栈中保存所有目前度为 0 的节点，每次弹出一个节点，然后修改与该节点相连的节点的度，若有新的度为 0 的节点出现，则再次放入栈中

```
#define MaxVertexNum 100     //图中顶点数目的最大值
typedef struct ArcNode{      //边表结点
  int adjvex;          //该弧所指向的顶点的位置
  struct ArcNode *nextarc;     //指向下一条弧的指针
  //InfoType info;       //网的边权值
}ArcNode;
typedef struct VNode{      //顶点表结点
  VertexType data;       //顶点信息
  ArcNode *firstarc;       //指向第一条依附该顶点的弧的指针
}VNode,AdjList[MaxVertexNum];
typedef struct{
  AdjList vertices;        //邻接表
  int vexnum,arcnum;       //图的顶点数和弧数
} Graph;              //Graph是以邻接表存储的图类型
```




```
bool TopologicalSort(Graph G){
  InitStack(S);        //初始化栈，存储入度为0的顶点
  for(int i=0;i<G.vexnum;i++)
    if(indegree[i]==0)
      Push(S,i);       //将所有入度为0的顶点进栈
  int count=0;         //计数，记录当前已经输出的顶点数
  while(!IsEmpty(S)){      //栈不空，则存在入度为0的顶点
    Pop(S,i);        //栈顶元素出栈
    print[count++]=i;      //输出顶点i
    for(p=G.vertices[i].firstarc;p;p=p->nextarc){
      //将所有i指向的顶点的入度减1，并且将入度减为0的顶点压入栈S
      v=p->adjvex;
      if(!(--indegree[v]))
        Push(S,v);     //入度为0，则入栈
    }
  }//while
  if(count<G.vexnum)
    return  false;       //排序失败，有向图中有回路
  else
    return  true;        //拓扑排序成功
}
```

前面提到，一个 AOV 网的拓扑排序可能有多个，这个多个序列的原因在于使用的队列或者栈，又或者你的代码的先后逻辑，到底是先弹出所有栈内节点，还是只要有度为 0 的节点放入栈。
YYK
2026/08/04 11:27

特备要注意，需要关注对环的存在的情况的讨论，本题中使用 coutn 的个数是否等于顶点个数来判别是否有环存在
YYK
2026/10/18 15:39

性能分析：

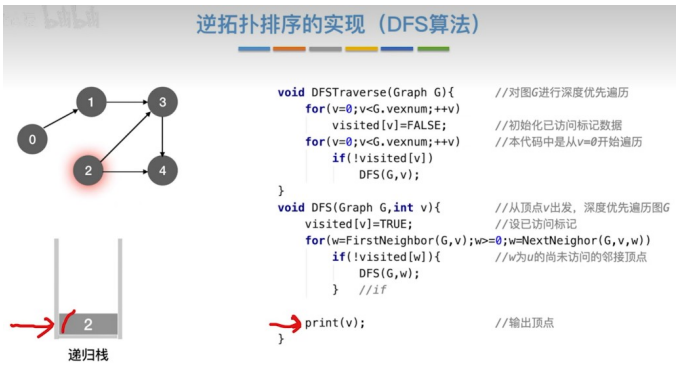
邻接表：O（V+E）、邻接矩阵 O（V^2）

拓扑排序和逆拓扑排序一样
YYK
2026/08/04 11:39

逆拓扑排序：

考虑到是一个节点的出度为 0

深度优先算法得到拓扑排序/逆拓扑排序：



比较重要，但我还没搞懂

YYK
2026/08/04 11:44

关键是

4.关键路径

AOE 网

这个 E 代表 edge ，即用边来表示一个一个的带权活动，适用于带权有向图。顶点代表的是事件。

事件代表的是状态，活动代表的是执行的过程

YYK
2026/08/04 11:45

（1）基本概念

e 代表 earlist、v 代表 event、l 代表 last、d 代表 **Total Slack** 总时差

①开始顶点（源点）、结束顶点（汇点）

②从源点到汇点有多条路径

③具有最大长度（带权值）的路径成为关键路径

④关键路径上的活动称为关键活动

活动时间余量 di=0 的叫关键活动

如果增长关键活动的工期，那么总时间一定增加；如果减少关键活动的工期，那么总时间可能减小，但是关键活动的工期减小到一定程度，它有可能变成非关键活动

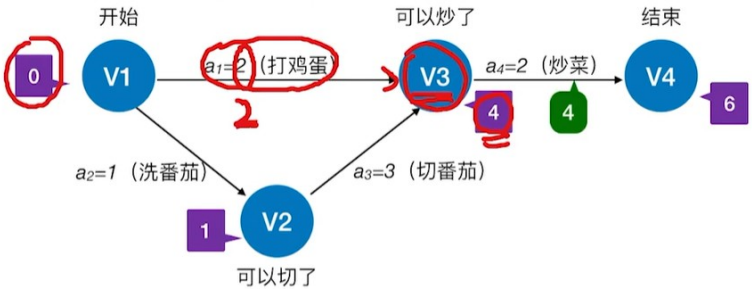
一个图中可能有多条关键路径，只有同时缩短所有关键路径上的时间，整个工期才会缩小

YYK
2026/08/04 14:39

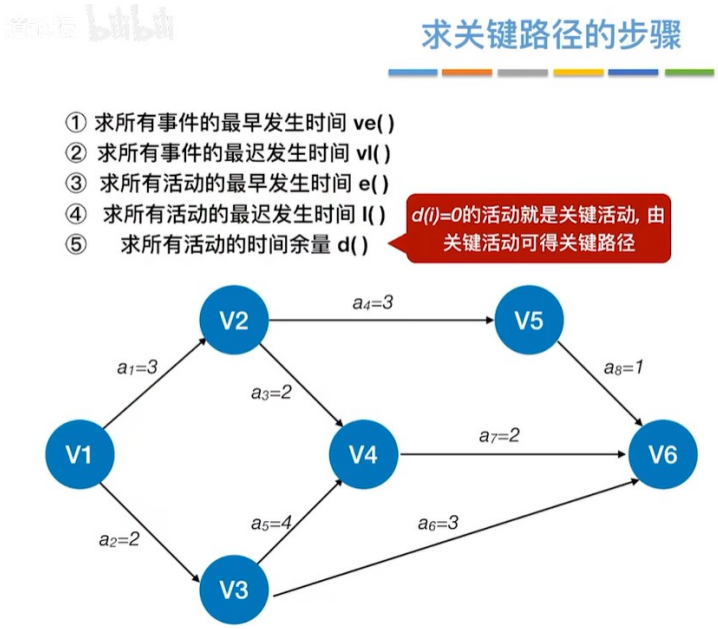
⑤事件的最早发生时间 ve（k），决定了某个依附于该事件的活动最早的开始时间

⑥活动的最早开始时间 e（i），该活动弧的起点的时间最早的开始时间

⑦根据事件的最迟开始时间减去活动所需要的时间，就可以得到某个活动最迟的开始时间



（2）例题



①求所有时间的最早发生时间 ve

YYK
2026/08/04 14:38

先对所有顶点进行拓扑排序，根据拓扑排序找到其与直接前驱事件时间+边活动时间的最大值。

拓扑序列：V1、V3、V2、V5、V4、V6

ve(1)=0

ve(3)=2

ve(2)=3

ve(5)=6

ve(4)=max{ve(2)+2, ve(3)+4}=6

ve(6)=max{ve(5)+1, ve(4)+2, ve(3)+3}=8

想一想为什么要取最大值呢？因为要保证该事件之前的所有活动都顺利完成之后，才达到该事件的状态
YYK
2026/08/04 14:42

②求所有事件的最迟开始时间

先对所有顶点进行逆拓扑排序，根据逆拓扑排序找到与其前驱节点事件-边活动时间的最小值

注意这里取的是最小值
YYK
2026/08/04 14:45

逆拓扑序列：V6、V5、V4、V2、V3、V1

vl(6)=8

vl(5)=7

vl(4)=6

vl(2)=min{vl(5)-1, vl(4)-2}=4

vl(3)=min{vl(4)-4, vl(6)-3}=2

vl(1)=0

这个 PPT 王道写错了一丢丢，应该是 vl（5）-3
YYK
2026/10/18 16:02

③所有活动的最早开始时间 e=弧尾节点的最早发生时间

④所有活动的最晚开始时间 l=头结点的最晚发生时间 vl-权重 weight

⑤活动余量 di=l-e

类型	元素（按顺序）	数值数组
ve（事件最早发生时间）	$V1, V2, V3, V4, V5, V6$	$[0, 3, 2, 6, 6, 8]$
vl（事件最迟发生时间）	$V1, V2, V3, V4, V5, V6$	$[0, 4, 2, 6, 7, 8]$
e（活动最早发生时间）	$a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8$	$[0, 0, 3, 3, 2, 2, 6, 6]$
l（活动最迟发生时间）	$a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8$	$[1, 0, 4, 4, 2, 5, 6, 7]$
d（活动时间余量）	$a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8$	$[1, 0, 1, 1, 0, 3, 0, 1]$

表 6.5　采用不同存储结构时各种图算法的时间复杂度

	Dijkstra	Floyd	Prim	Kruskal	DFS	BFS	拓扑排序	关键路径
邻接矩阵	$O(n^2)$	$O(n^3)$	$O(n^2)$	-	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$
邻接表	-	-	-	$O(e\log_2 e)$	$O(n+e)$	$O(n+e)$	$O(n+e)$	$O(n+e)$

5.有向无环图 DAG 描述表达式

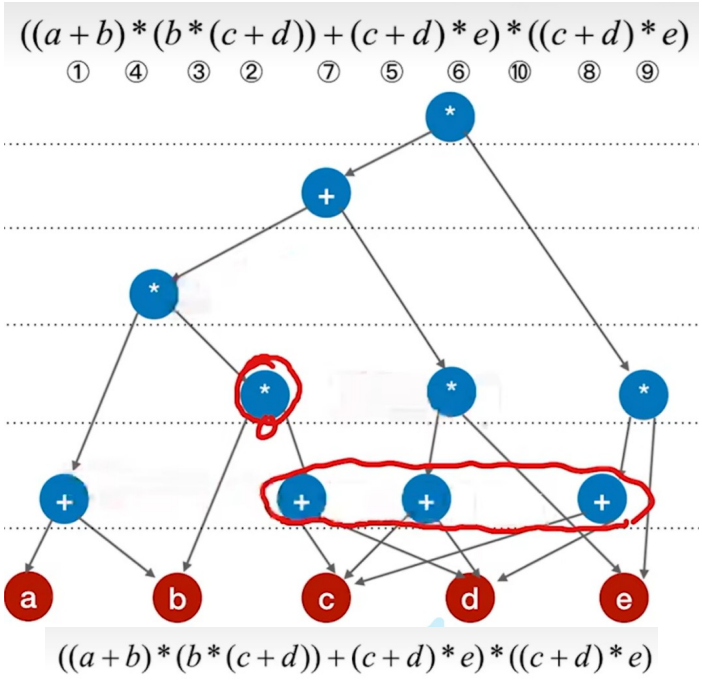
可以使用咸鱼自创算法：

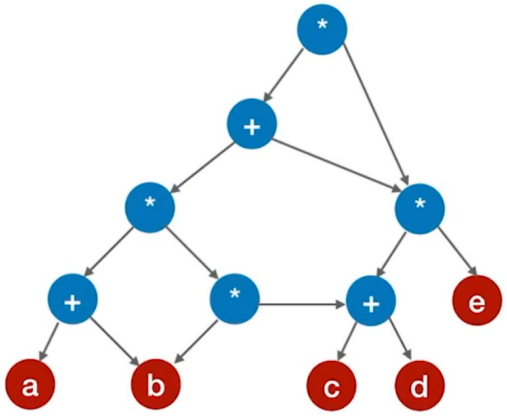
①先根据算术方程式，写出各个符号的运算次序

②写出所有的不重复的运算数，放在最下面一层

③依次将各个操作符写入树中

④遇到相同操作数和相同操作符，则合并


((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)
① ④ ③ ② ⑦ ⑤ ⑥ ⑩ ⑧ ⑨
*
+
*
*
*
*
+
+
+
+
a
b
c
d
e
((a+b)*(b*(c+d))+(c+d)*e)*((c+d)*e)


*
+
*
*
+
*
+
e
a
b
c
d

六、查找

(一)查找的基本概念

平均查找长度 ASL=∑(每个数据元素查找到的概率*该数据查找的次数)

一般认定每个数据查找到的概率相等，都为 1/n

YYK
2026/08/05 10:53

(二)顺序查找法

适用于链表+顺序表

(1)一般线性表顺序查找

从头到尾一次扫描。

ASL 成功=(n+1)/2 次(保证每个数据查找到的概率相等为 1/n)

ASL 失败=n+1

成功查找的 ASL 与元素排列顺序无关，所以无论是有序还是无序，那么顺序成功查找的 ASL 都一样

ASL 成功 = (1+2+3+4....+n)*1/n=(n+1)/2
而且，平均查找长度就是每个元素的查找长度 * 概率，最后不需要除以所有元素的个数

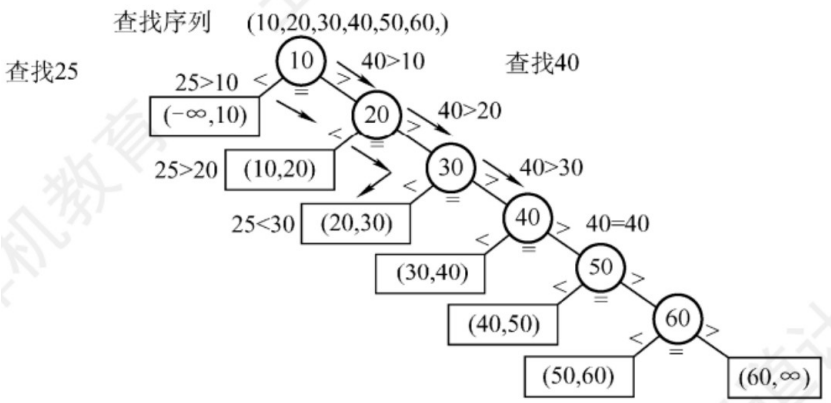
YYK
2026/08/05 14:35

(2)有序线性表顺序查找

判定树：(矩形节点成为失败节点)

特点是若查找到了某个比目标数更大的数，则无需后续遍历，直接停止，节省时间

YYK
2026/08/05 11:02



ASL 成功=(n+1)/2(保证每个数据查找到的概率相等为 1/n)

$$ASL_{不成功}=\sum_{j=1}^{n}q_j(l_j-1)=\frac{1+2+\cdots+n+n}{n+1}=\frac{n}{2}+\frac{n}{n+1}$$

(三)分块查找法 (索引顺序查找)

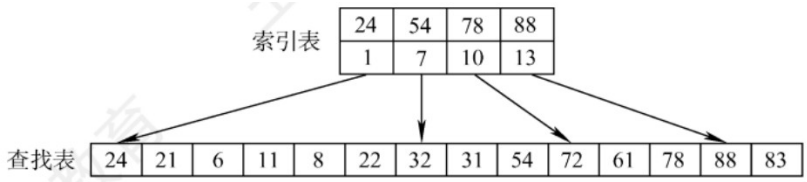


图 7.3　分块查找示意图

先查找索引表：索引表按照顺序排列，每个索引表指向查找表中的一个位置，如被查找的数小于索引表的某个数，那就去该索引表对应的查找表中查找数据。

查找表中的各个分块内的数据大小无序，但块间的大小有序

	成功	失败
索引表为顺序查找	$ASL=L_1+L_S=\frac{b+1}{2}+\frac{s+1}{2}=\frac{s^2+2s+n}{2s}$ 此时，若 $s=\sqrt{n}$，则平均查找长度取最小值 $\sqrt{n}+1$。	好像算不出来。。。
索引表为二分查找	ASL 成功=折半+(s+1)/2	

其中 b=n/s，这里是把每块内的数据元素个数固定统一了，只有这样才能保证平均查找长度最小为
YYK
2026/08/06 10:11

区分一下折半查找和二叉排序树的区别。
折半查找是平衡二叉树，可不是二叉排序树，二叉排序树可以是一条只有右孩子的“链”，这也就是它的最坏的查找时间为 O（ n ），最好的查找时间跟折半查找一样

而折半查找的平均查找长度和最坏情况的都是 O（ $\log_2 n$ ）
YYK
2026/08/05 14:52

(四)折半查找法

折半查找（有序+顺序表，且每次取整的方式是一样的）

①基本查找过程

基本查找过程是非常重要的，有一个容易忽略的地方，就是下一次的范围是 mid+1，或者 mid-1 为端点的，不是以 mid 为端点

```
 7  10  13  16  19  29  32  33  37  41  43
 ↑low                ↑mid                ↑high
```

第一次查找时，将中间位置元素与 key 比较。因为 11＜29，说明待查元素若存在，则必在范围[low,mid-1]内，令 high 指向位置 mid-1，high=mid-1=5，mid=(1+5)/2=3，第二次查找范围为[1,5]。

```
 7  10  13  16  19  29  32  33  37  41  43
 ↑low    ↑mid    ↑high
```

折半查找的过程是依据于数据元素的大小关系来比较得出的取左区间和右区间，而不是根据下标！自己在做 2013 真题的时候进入了这个误区，为了追求更短的 ASL，忘记了折半查找的最关键前提就是顺序+有序，自己擅自修改了表内元素的顺序，折半查找也就失去了效果！
YYK
2026/08/18 21:51

我在比较完 mid 之后，选择了下一个小范围的区间，其中 mid 已经比较过了，要根据情况进行加一减一操作
YYK
2026/08/05 14:59

②折半查找判定树

23.【2017 统考真题】下列二叉树中，可能成为折半查找判定树（不含外部结点）的是（ ）。

A.



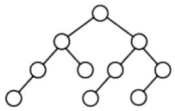
B.



C.



D.



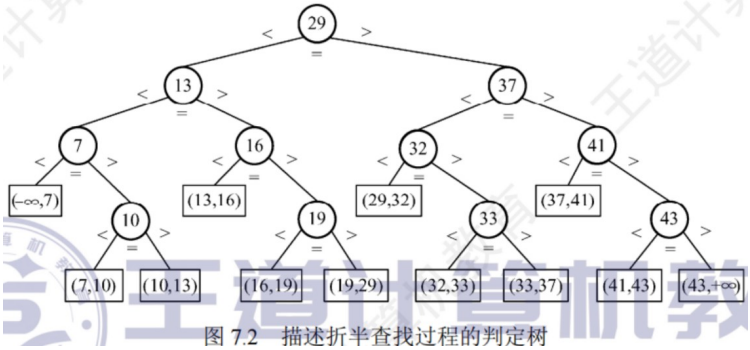


图 7.2　描述折半查找过程的判定树

查找成功的平均查找长度为

$$ASL=\frac{1}{n}\sum_{i=1}^{n}l_i=\frac{1}{n}(1\times1+2\times2+\cdots+h\times2^{h-1})=\frac{n+1}{n}\log_2(n+1)-1\approx\log_2(n+1)-1$$

时间性能为 O（$\log_2 n$）

查找失败的平均查找长度为

ASL 失败=（每一层失败节点的个数*该层的层号，并求和）/n+1

判定树的高度不包括下面的叶子结点（错误节点）
判定树的最大高度为 $\log_2$（n+1）向上取整，所以最大的比较次数=最大高度

折半查找判定树还有一个比较重要的元素：任何一个节点的左子树和右子树的节点个数是固定差距的（永远左比多一个或相等、永远右比左多一个或相等，这涉及到取整的策略）！不可能一会左比右多，一会右比左多（2017 真题）这是取决于取整策略导致的，向下取整，那么左区间就会比右区间少一个数，此后左区间永远小于等于右区间
YYK
2026/08/06 09:45

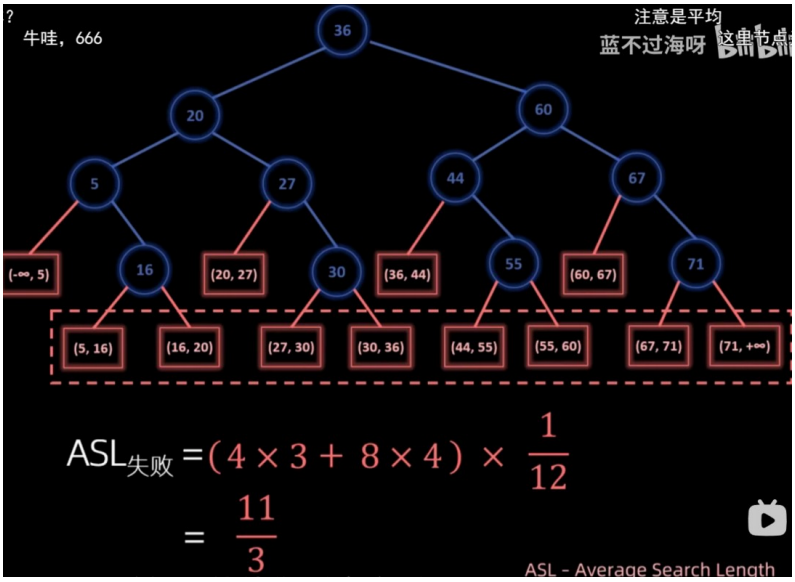
折半查找的判定树的构造过程要根据实际的数据，查找的过程进行构造，不能凭空构造
YYK
2026/08/06 10:45

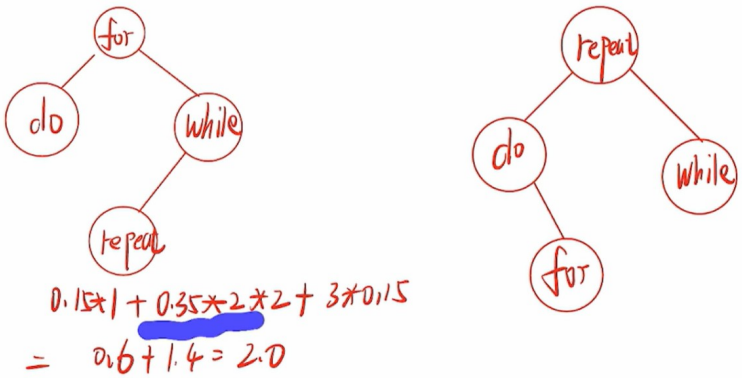
这个树一定是平衡二叉树，因为折半查找的查找过程严格遵循平衡二叉树的定义，即左右子树的深度差不超过 1，且左右子树均为平衡二叉树
YYK
2026/08/05 14:23

注意前面这个 1*1+2*2+3*4+4*8.... 这个才是这个公式原有的味道！
YYK
2026/08/05 11:36

这里指的是矩形失败节点的父节点圆圈的层号，矩形节点是不计入比较次数的
YYK
2026/08/06 10:01

这里的 n 代表总共有多少个数
YYK
2026/08/05 11:38





来自于 2013 年大题，折半查找判定树，

YYK
2026/10/26 21:23

(五)树型查找

1.二叉树搜索树

(1)二叉排序树的定义：左 < 根 < 右

(2)二叉排序树的查找：

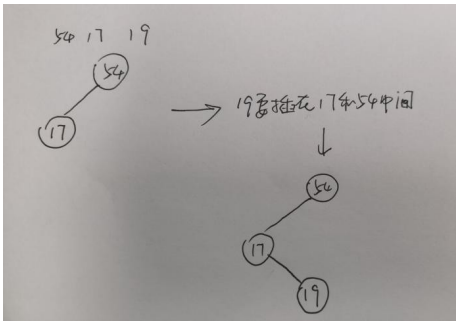
从根节点出发，若比较的树小于根节点，则向左子树出发，否则向右子树，直

到遇到想要找的数或者子树为空

（3）二叉排序树的插入：

一个一个插入，新插入的树要满足原来的树的范围，不要重新构造！

（4）二叉排序树的构造：

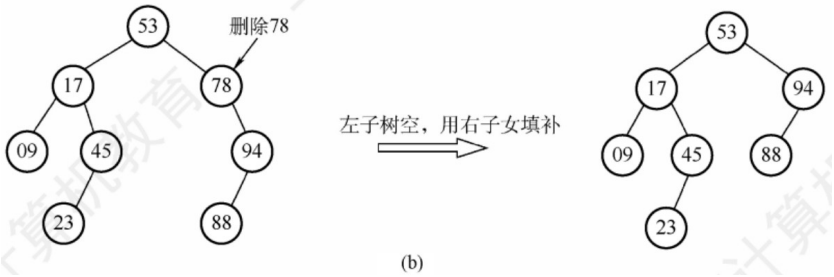


一组相同的数据，由于插入顺序不同，那么二叉排序树也就可能不同
YYK
2026/08/06 11:59

（5）二叉排序树的删除的三种情况：

①删除叶节点 easy

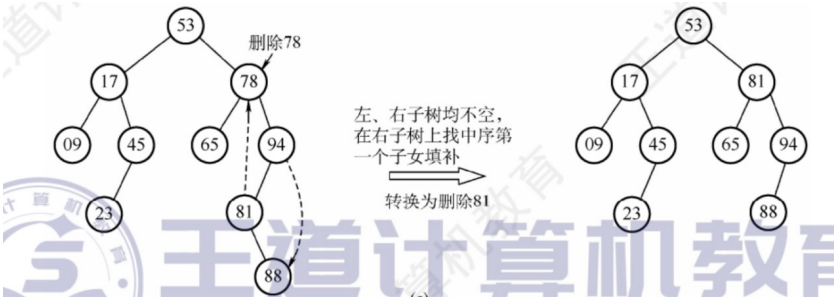
②该节点只有一个左子树或者一个右子树，则让其子树代替它即可



(b)

③该节点同时有左子树和右子树，则让它的直接后继/直接前驱替代它

前去后继都可以
YYK
2026/11/03 09:58



删除一个节点再插入它，前后的树的形状可能是不同的（2013真题）
先插入一个节点再立即删除它，前后树的形状不变！

YYK
2026/08/06 11:49

（6）二叉排序树的查找效率分析：

①二叉排序树的查找效率取决于树的高度，如果是平衡二叉树则查找长度与O（$\log_2 n$）成正比，若只有右子树没有左子树，即一条链，则平均查找长度为（n+1）/2

②二叉排序树的插入删除无需移动节点，最主要的就是查找的过程O（$\log_2 n$），而顺序表中插入删除需要移动元素，代价为O（n）

2.平衡二叉树

（1）定义：

左子树与右子树的高度差称为平衡因子-1；0；1

平衡二叉树可以是空树

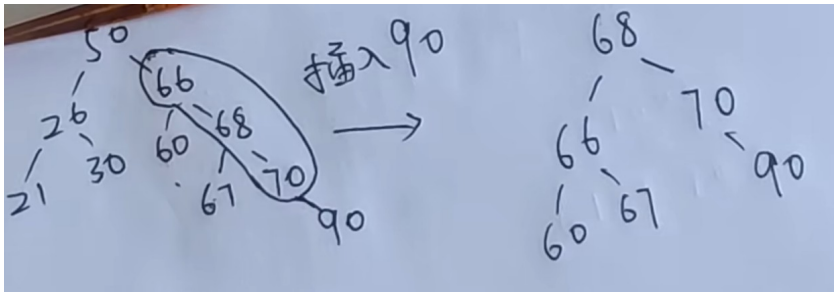
插入前后中序遍历要保持一致

（2）平衡二叉树的插入

①先插入

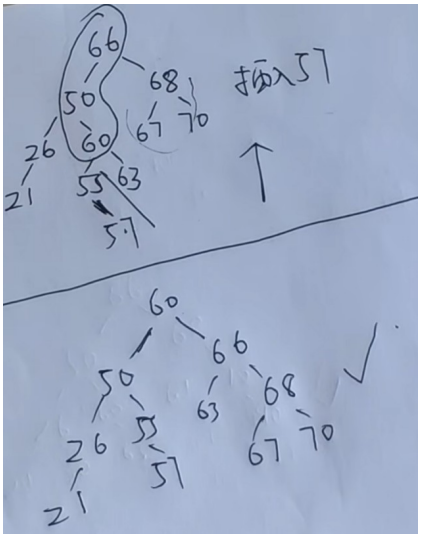
②向上找到低最小不平衡子树的根节点

③从该根节点出发沿着刚才的路径，连续圈出三个节点，将这三个节点重新调整



来自 b 站 up 主“公台所言极是”，完爆王道的左旋右旋

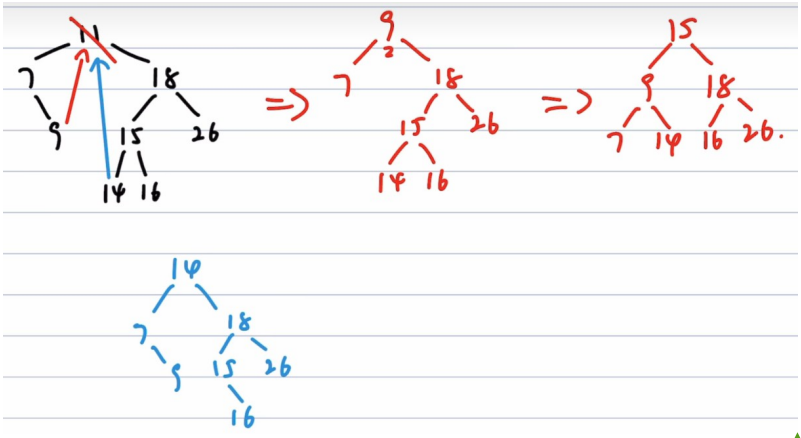
YYK
2026/08/06 14:43



（3）平衡二叉树的删除

①删除叶节点，直接删除叶节点之后，调整平衡即可

②删除非叶节点，把它的直接前驱或者直接后继补上去，然后调整平衡即可



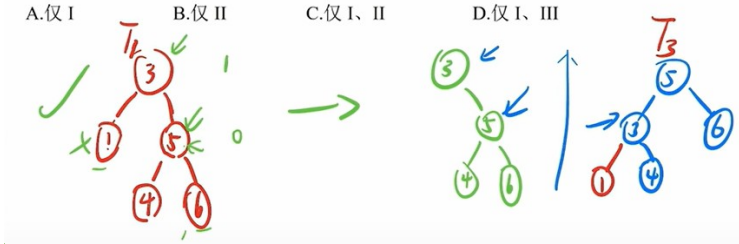
25.【2019 统考真题】在任意一棵非空平衡二叉树(AVL 树)T1 中，删除某结点 v 之后形成平衡二叉树 T2,再将 v 插入 T2 形成平衡二叉树 T3。下列关于 T1 与 T3 的叙述中，正确的是(　　)

I.若 v 是 T1 的叶结点，则 T1 与 T3 可能不相同

II.若 v 不是 T1 的叶结点，则 T1 与 T3 一定不相同

III.若 v 不是 T1 的叶结点，则 T1 与 T3 一定相同

A.仅 I　　B.仅 II　　C.仅 I、II　　D.仅 I、III



这里注意找的是不平衡那一侧的路径的相连三个节点，例如本题中的 9 、 18 、 15

YYK
2026/08/06 14:49

这个不平衡路径可以是 356 或者 354 都行

YYK
2026/11/05 17:56

（4）平衡二叉树的查找

①查找次数不超过树的深度 $\log_2 n$,所以平均查找效率为 O（$\log_2 n$）

②平衡二叉树各个高度最小的节点数 n_h

$n_0=0;n_1=1;n_2=2$

$$n_h = n_{h-2} + n_{h-1} + 1,$$

③深度为 h 的平衡二叉树的最多节点数为满二叉树

解得：**最小深度为** $\lceil \log_2(n+1) \rceil$。

平衡二叉树的深度公式，一定要谨记！

YYK
2026/08/07 11:45

14. 在平衡二叉树的基本操作中，可能发生两次旋转的操作是（ ）。

A. 添加、删除结点 B. 仅删除结点 C. 仅添加结点 D. 都不会

A

YYK
2026/08/08 16:36

左根右，根叶黑，不红红，黑路同

YYK
2026/11/03 10:01

3.红黑树

（1）定义：

红黑树是在平衡二叉树需要频繁调整结构的基础上，引入的新的数据结构，本质上也是一个平衡搜索树。

① 每个结点或是红色，或是黑色的。
② 根结点是黑色的。
③ 叶结点（虚构的外部结点、NULL 结点）都是黑色的。
④ 不存在两个相邻的红结点（红结点的父结点和孩子结点均是黑色的）。
⑤ 对每个结点，从该结点到任意一个叶结点的简单路径上，所含黑结点的数量相同。

www.cskaoyan.co

我们都知道二叉查找树（BST）支持插入、删除、查找操作，理想情况下时间复杂度是 O(log n)，效率很高。但 BST 在频繁插入、删除之后，很容易变得“歪脖子”，甚至退化成链表，时间复杂度直接掉到 O(n) ……

那为什么不直接用 AVL 树呢？
AVL 是“强迫症患者”，每次插入删除都可能引发多次旋转，代价太高；
而红黑树是“实用主义者”，它不要求完全平衡，只要能保证查找效率不崩就行。
平衡二叉树在查询上稍微高效一点，但是在插入和删除上逊于红黑树（插入删除旋转操作比较少）

一句话总结：
红黑树是在“速度”、“稳定性”和“实现成本”之间找到的一个折中王者！

YYK
2026/08/07 09:51

红黑树不需要满足平衡因子仅限于-1、0、1 的特性

一棵含有 n 个结点的红黑树的高度至多为 $2\log_2(n+1)$

属于自平衡树（指的是在插入和删除过程中会动态调整保持平衡）

红色节点最多是黑色节点的两倍（根节点为黑、两个叶子结点为红）

若红黑树为全黑节点，则一定是满二叉树

（2）插入操作

①插入是根节点→直接变黑（别忘记了本质上还是二叉排序树）

这主要就是第一个节点的插入操作

②插入节点的叔叔节点是红色节点：

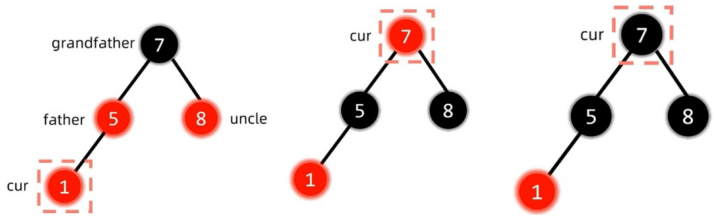
调整爷爷、叔叔、爸爸节点，红变黑、黑变红，让爷爷变成插入节点，继续判定爷爷是否违反规则

左根右、根叶黑、不红红、黑路同（强调所有节点的子路径）
长路径不超过短路径的两倍
n 个内部节点 1 的红黑树高度为 $2\log_2(n+1)$
高 h 的红黑树的最小节点数 $=2^h-1$，最多 $2^{2h}-1$
插入节点默认为红色

YYK
2026/08/07 09:49

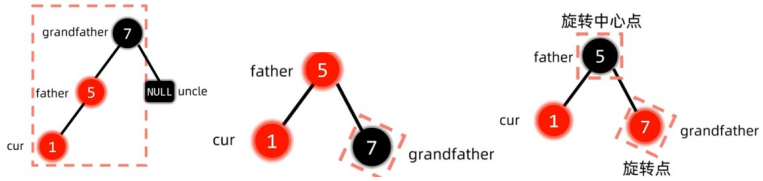
红黑树插入操作最多两次旋转，删除操作最多三次旋转

YYK
2026/08/08 16:40

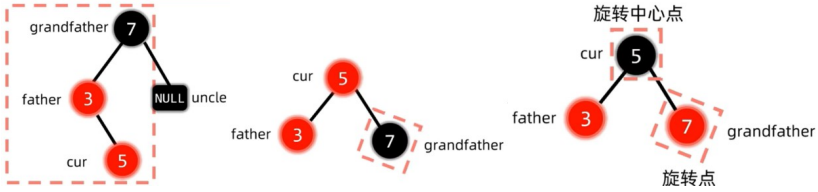


③插入节点的叔叔是黑色节点：旋转操作然后变色（旋转的两个节点调色）

LL：



LR:



红黑树 VS AVL 树：谁更强?

特性	AVL 树	红黑树
平衡性	非常严格	近似平衡
插入性能	较慢（多次旋转）	快（最多两次旋转）
删除性能	更慢（多次旋转）	较快（最多三次旋转）
查找性能	更快	稍慢（但仍为 O(log n)）

红黑树是一种自平衡二叉树，但不是严格的平衡二叉树

YYK
2026/11/03 10:24

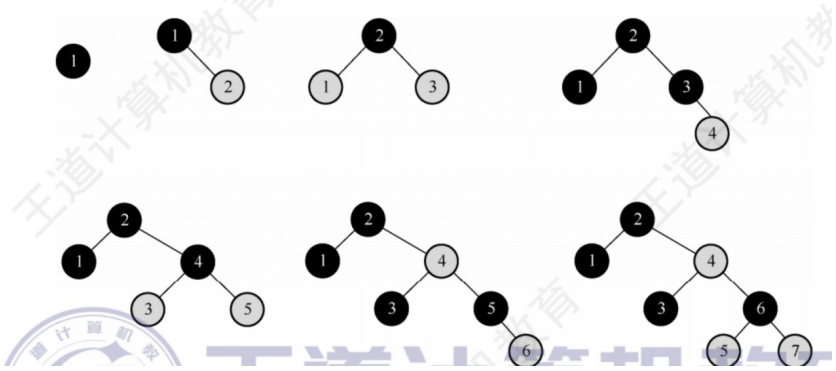
AVL 树适合读多写少的场景，红黑树更适合写多读少、需要稳定性能的场景。

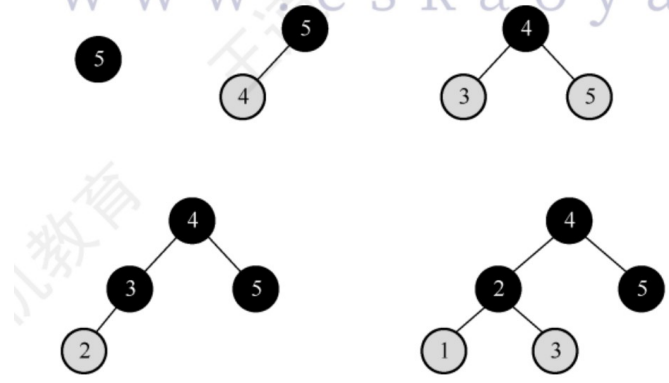
插入删除操作的时间复杂度都为 O（$log_2 n$）

YYK
2026/08/07 10:04

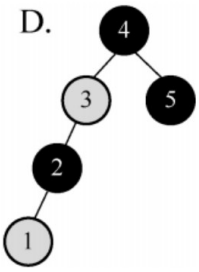
（3）练习操作

关键字 1, 2, 3, 4, 5, 6, 7 依次插入红黑树后的形态变化如下：





D.



①关键是插入五的这一步，插入后发现 45 红，且 5 的叔叔不存在（黑），那么就旋转操作；

②插入 6 发现， 6 的叔叔为红，那么爷爷叔叔爸爸全都变色，再检查爷爷是否违背

③插入⑦发现 67 红， 7 的叔叔不存在（黑），则旋转操作
YYK
2026/08/07 14:54

插入③后发现③的叔叔不存在（黑），那么就需要先旋转，让④当根节点，然后⑤当子节点，同时调换④⑤的颜色！！！非常重要！

否则为什么不是三四五同时都是黑色的呢？一定要遵循规则！
YYK
2026/08/07 15:08

这就不是一个红黑树，自己及其容易忽略的地方就是，所有节点的子路径都需要符合黑路同的定义，显然③的左右子路径不满足；还有根节点必须是黑色的，所以 1 没有被置换为黑色
YYK
2026/08/07 15:04

(六)B 树及其基本操作、B+树的基本概念

(1)B 树定义

①平衡因子均为 0 的 m 路平衡查找树，可以为空树，但 m 路树的子节点内最多存储 n-1 个数

② 3）除根结点外的所有非叶结点至少有⌈m/2⌉棵子树，即至少有⌈m/2⌉−1 个关键字。

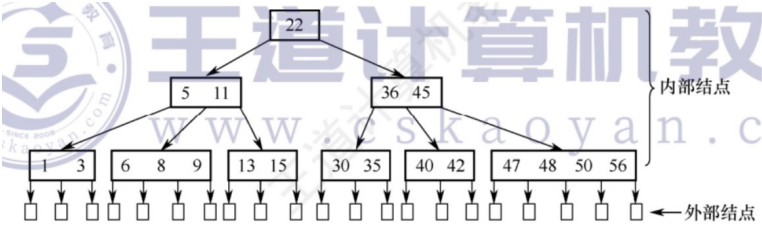


图 7.28　一棵 5 阶 B 树的实例

③孩子的个数等于该节点内元素的个数+1

④根节点存在的话，里面必须存在关键字

⑤ B 树的高度不包含最后一层失败节点

（2）B 树的查找

①搜索整个 B 树是在磁盘上查找（因此在该树上的深度，决定了查找效率）

②节点内部的数的查找是把该节点读入内存，在内存中顺序查找或者折半查找

B 树的关键是减少磁盘 IO 次数，相对于平衡二叉树、红黑树、二叉查找树来说，B 树的高度更低；根节点常驻内存，然后通过根节点来查找各个磁盘块，磁盘块内部存储的有各个下一个分区的指针

YYK
2026/11/03 10:41

（3）B 树的高度

若 $n \geqslant 1$，则对任意一棵包含 n 个关键字、高度为 h、阶数为 m 的 B 树：

1）若让每个结点中的关键字个数达到最多，则容纳同样多关键字的 B 树的高度达到最小。因为 B 树中每个结点最多有 m 棵子树，$m-1$ 个关键字，所以在一棵高度为 h 的 m 阶 B 树中关键字的个数应满足 $n \leqslant (m-1)(1+m+m^2+\cdots+m^{h-1}) = m^h - 1$，因此有

$$h \geqslant \log_m(n+1)$$

2）若让每个结点中的关键字个数达到最少，则容纳同样多关键字的 B 树的高度达到最大。第一层至少有 1 个结点；第二层至少有 2 个结点；除根结点外的每个非叶结点至少有⌈m/2⌉棵子树，则第三层至少有 2⌈m/2⌉个结点……第 $h+1$ 层至少有 $2(\lceil m/2 \rceil)^{h-1}$ 个结点，注意到第 $h+1$ 层是不包含任何信息的叶结点。对于关键字个数为 n 的 B 树，叶结点即查找不成功的结点为 $n+1$，由此有 $n+1 \geqslant 2(\lceil m/2 \rceil)^{h-1}$，即 $h \leqslant \log_{\lceil m/2 \rceil}((n+1)/2)+1$。

例如，假设一棵 3 阶 B 树共有 8 个关键字，则其高度范围为 $2 \leqslant h \leqslant 3.17$，取整数。

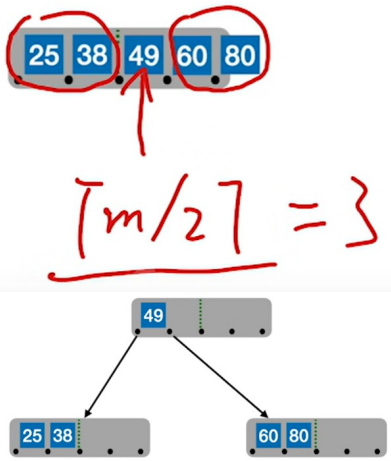
（4）B 树的插入

①首先铭记 m 阶 B 树

3）除根结点外的所有非叶结点至少有⌈m/2⌉棵子树，即至少有⌈m/2⌉−1 个关键字。 ，最多有 m-1 个关键字

这个取值范围是构造 B 树的关键之处

YYK
2026/08/08 17:41





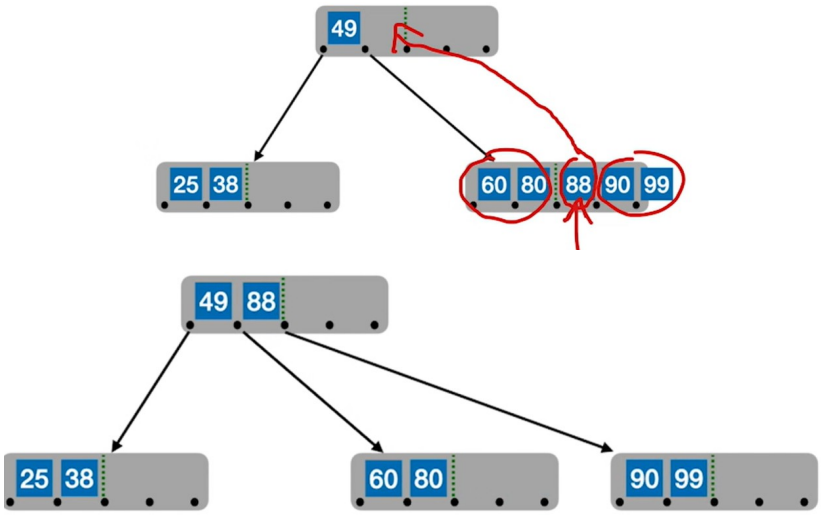
学会基础操作，找分裂的位置！

YYK
2026/08/08 17:43

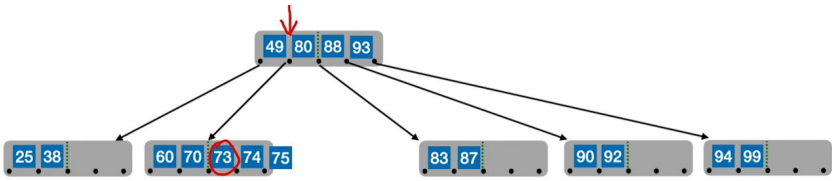
在插入key后，若导致原结点关键字数超过上限，则从中间位置（$\lceil m/2 \rceil$）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（$\lceil m/2 \rceil$）的结点插入原结点的父结点

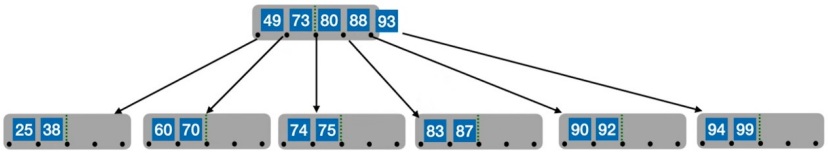
②每次插入的新元素一定是在最底层的终端节点，否则如果插入到根节点，那么失败节点

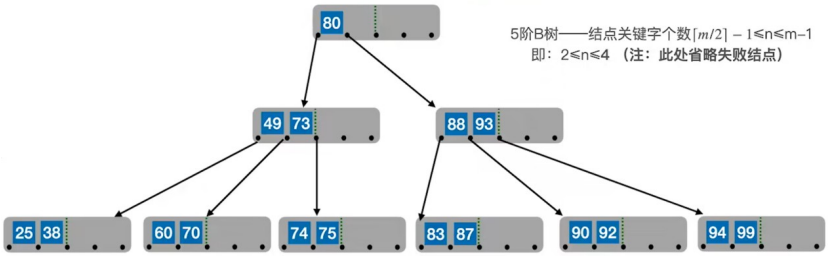
就看不位于同一层了，不满足 B 树的定义



③如果根节点也满了，那么根节点需要向上分裂

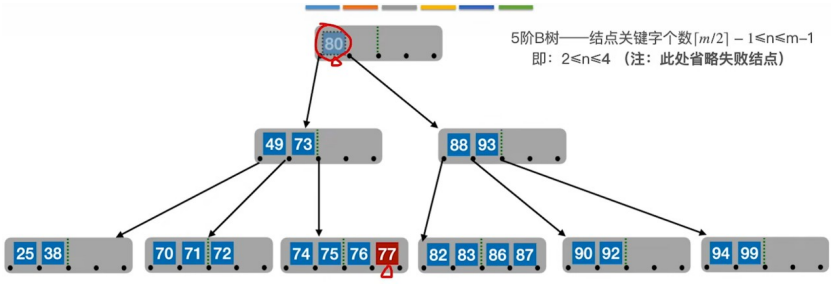




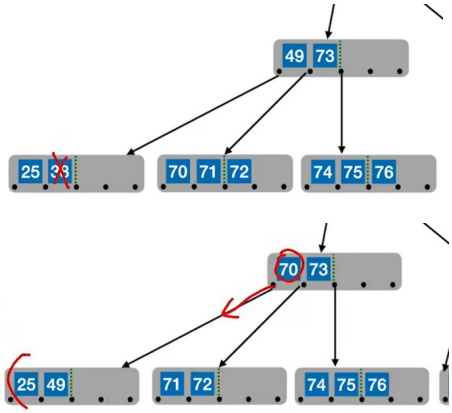


（5）B 树的删除

①若删除节点的元素为非终端节点，则找直接前驱（或后继）顶替它（删除 80,由 77 顶替它）

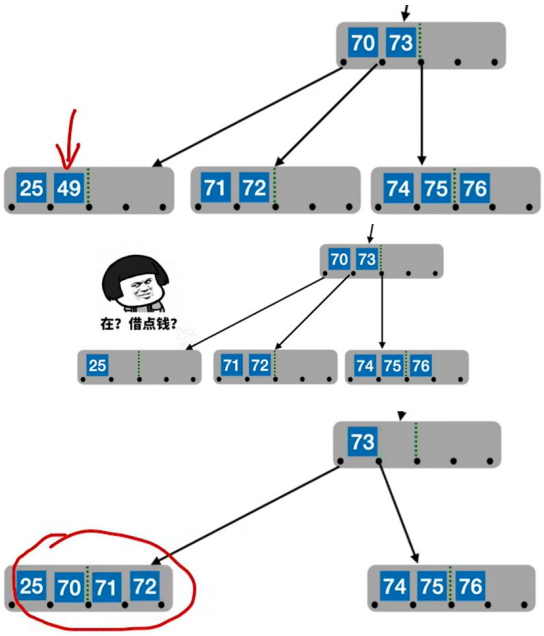


②删除节点内的元素低于最低标准，则先看兄弟够借不，够借就让父亲下来一个元素，让兄弟节点上去一个元素

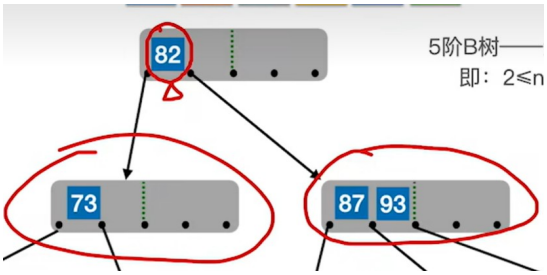


③删除节点内的元素低于最低标准，兄弟也不够借，则把两个兄弟合并，然后同时也要把

父亲节点的某个元素也合并进来



可以注意到到这里结束的话，它的双亲节点也不够用了！所以接下来进行的就是把这个双亲节点和其兄弟继续进行借出？合并？如下图
YYK
2026/08/08 20:24



(1) B+树的定义

B+树是应数据库所需而出现的一种 B 树的变形树。

一棵 m 阶 B+树应满足下列条件：

1）每个分支结点最多有 m 棵子树（孩子结点）。

2）非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。

3）结点的子树个数与关键字个数相等。

4）所有叶结点包含全部关键字及指向相应记录的指针，叶结点中将关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接起来（支持顺序查找）。

5）所有分支结点（可视为索引的索引）中仅包含它的各个子结点（下一级的索引块）中关键字的最大值及指向其子结点的指针。

(2) B+树与 B 树的差异化分析

m 阶 B+树与 m 阶 B 树的主要差异如下：

1）在 B+树中，具有 n 个关键字的结点只含有 n 棵子树，即每个关键字对应一棵子树；而在 B 树中，具有 n 个关键字的结点含有 $n+1$ 棵子树。

2）在 B+树中，每个结点（非根内部结点）的关键字个数 n 的范围是 $\lceil m/2 \rceil \leqslant n \leqslant m$（非叶根结点：$2 \leqslant n \leqslant m$）；而在 B 树中，每个结点（非根内部结点）的关键字个数 n 的范围是 $\lceil m/2 \rceil - 1 \leqslant n \leqslant m-1$（根结点：$1 \leqslant n \leqslant m-1$）。

3）在 B+树中，叶结点包含了全部关键字，非叶结点中出现的关键字也会出现在叶结点中；而在 B 树中，最外层的终端结点包含的关键字和其他结点包含的关键字是不重复的。

4）在 B+树中，叶结点包含信息，所有非叶结点仅起索引作用，非叶结点的每个索引项只含有对应子树的最大关键字和指向该子树的指针，不含有对应记录的存储地址。这样能使一个磁盘块存储更多的关键字，使得磁盘读/写次数更少，查找速度更快。

5）在 B+树中，用一个指针指向关键字最小的叶结点，将所有叶结点串成一个线性链表。

①假设 B 树的子节点最多有四个元素，那么就会被分成五个区间，这五个区间就意味着这个树是五路 B 树；但是 B+ 树的子节点里最多有四个元素，每个元素代表一个区间（不大于该数的区间），所以最多四个区间，这就成为四阶 B+ 树

② B+ 树中的元素个数的计算方法和 B 树不一样，特别是 B+ 树对于非根节点内部、非叶节点 & 非根节点不一样

③ B+ 树中的非叶节点只起到索引作用，并没有获取最终结果的作用

YYK
2026/08/08 20:39

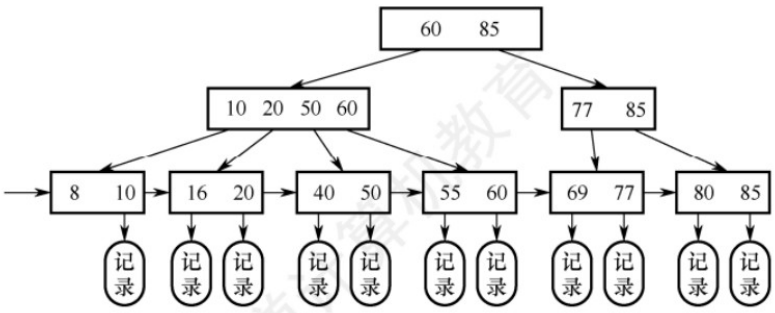


图 7.32　B+树结构示意图

B+ 树的查找有两种方式：一种是通过叶子结点直接扫描（相当于顺序查找）；另一种是通过根节点进行查找
YYK
2026/08/08 20:45

(七)散列(Hash)表

1.散列表的构造方法

不包含链地址法的散列表，确实只能基于连续存储空间（数组）实现，且核心依赖数组的随机访问能力。但是加入了链地址法，就可以利用链表的顺序结构了
YYK
2026/08/09 16:07

①定义域必须包含所有关键字、值域的范围依赖于散列表大小

②尽可能均匀分布、尽可能减少冲突

③散列函数尽量简单

（1）直接定址法：H=a*key+b（若关键字分布不连续，造成哈希表存储空间浪费）

（2）除留余数法：H=key%p（p 是最近哈希表长的质数，要尽量减少冲突）

（3）数字分析法：适用于先已知关键字集合，再构造哈希表

（4）平方取中法：取关键字的平方的中间几位（适用于没有规律，分布散乱的关键字）

注意平方探测法和平方取中法的区别，一个是解决冲突策略，一个是散列函数的制定方法

：在数据规模较小的情况下，平方取中法能有效生成散列地址，不会因数据过多导致冲突急剧增加。比如在小型数据库或特定的局部数据处理中，可使用该方法进行散列。
：当关键字取值范围较广且无明显规律时，平方取中法效果较好。它先对关键字平方，再取中间若干位作为散列地址。例如关键字为 45，平方后是 2025，若取中间两位 02 作为散列地址，能将不同关键字映射到相对分散的地址空间，减少冲突。

YYK
2026/08/09 16:29

2. 处理冲突的方法

（1）开放定址法

①线性探测（也叫线性探测再散列，408 中通常这么叫）：往后延伸即可（造成大量元素在相邻地址堆积、降低了查找效率）

逻辑删除 vs 物理删除
采用开放定制法时，不能随意删除元素，否则导致后续的查找“断链”，合理的方法是设置一个标记位，标记该位置数据已被删除（这叫逻辑删除），但是也会造成虽然被删除，实际上占着表的空间，表满了。
YYK
2026/08/08 22:05

②平方探测：散列表长度必须是一个可以表示成 4k+3 的素数，k 小于等于 m/2（缺点是不

能探测到所有单元，但至少可以探测到一半的单元，d 的取值是 0、1、-1、4、-4、9、-9...

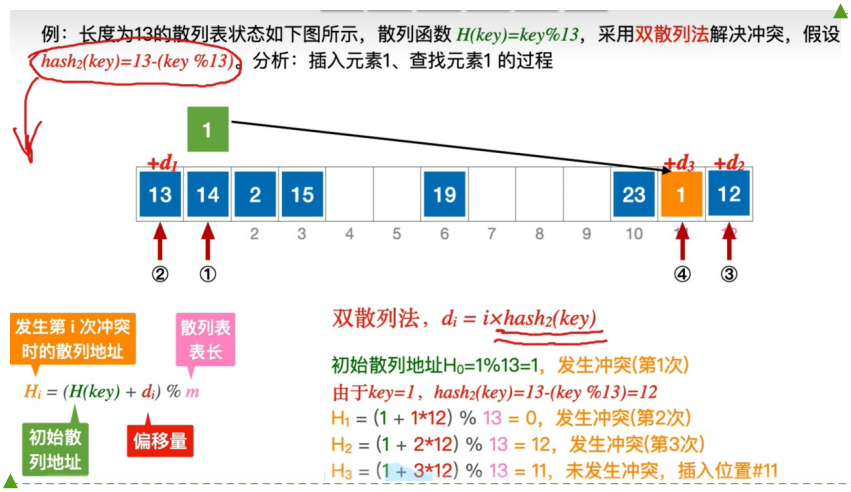
这是有顺序的，不能乱！）

③双散列法：

双散列法。$d_i = i \times \text{Hash}_2(\text{key})$。需要使用两个散列函数，当通过第一个散列函数 $H(\text{key})$得到的地址发生冲突时，则利用第二个散列函数 $\text{Hash}_2(\text{key})$计算该关键字的地址增量。它的具体散列函数形式如下：

$$H_i = (H(\text{key}) + i \times \text{Hash}_2(\text{key})) \% m$$

初始探测位置 $H_0 = H(\text{key}) \% m$。i 是冲突的次数，初始为 0。



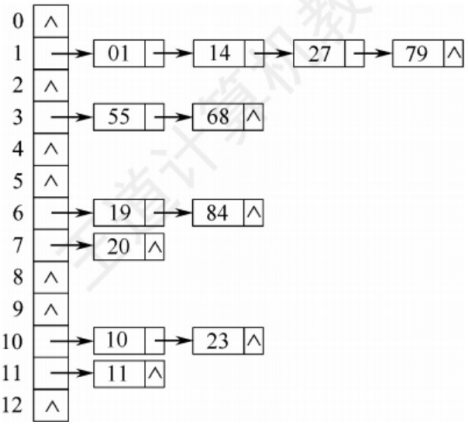
两个哈希函数都要牢牢记住，特别是第二个，甚至可以解决多次冲突
YYK
2026/08/08 22:02

关键在于要设计第二个散列函数！
YYK
2026/11/03 11:13

④伪随机法：伪随机序列

（2）拉链法

采用头插法（尾插法也可以，具体看题目要求），对于同一冲突的关键字，建立一个链表



3. 散列查找及其性能分析的应用

①查找次数=比较次数

②失败的比较次数=从当前位置一直到最后的查找失败的位置

③平均查找长度=所有内容的查找次数之和/关键字的个数

这里是关键字的个数，而不是散列表的长度，一定要想明白是为什么
YYK
2026/08/09 16:01

④装填因子 α=表中记录的关键字个数/散列表长度（装填因子越大，则发生冲突的概率越大）

08. C

散列表的平均查找长度与装填因子α直接相关，表的查找效率不直接依赖于表中已有表项个数n或表长m。若表中存放的记录全是某个地址的同义词，则平均查找长度为$O(n)$而非$O(1)$。

意思就是：如果没有同义词，则效率就是 O（1），但是存在了同义词了，就需要考虑一下同义词的个数和偏离地址的长度，所以平均查找长度可能不是 O（1）了，但是也不直接依赖于元素的个数
YYK
2026/08/09 16:12

4. 散列查找失败计算方法

21. 【2019 统考真题】现有长度为 11 且初始为空的散列表 HT，散列函数是 H(key)=key%7，采用线性探查（线性探测再散列）法解决冲突。将关键字序列 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是（ ）。

A. 4　　B. 5.25　　C. 6　　D. 6.29

需要注意的几个坑：假设 0-10 号位置

① 7-10 号位置是无法直接散列到的，所以不需要从这些位置出发考虑查找失败的长度

②平均查找长度是指从 0-6 号位置散列后，找不到该数的查找次数，所以最后要除以散列

表可以散列的数字个数 7

(八)字符串模式匹配

(九)查找算法的分析及应用

七、排序

(一)排序的基本概念

(1)算法的稳定性

不改变相同元素的前后顺序，算法的稳定性与优劣无关

(2)为什么要有外部排序

数据太多，无法全部装入内存，每次只能读入一块，进行排序

③在链表上无法实现排序算法

(二)直接插入排序

(1)算法思想

每次选择一个未排序的数字，将其插入到前面排好序的序列当中，移动位置即可



(2)算法性能

空间	只占用常数空间，故为 O(1)
最好时间	整体有序，扫描一遍即可，故为 O(n)
最坏时间	整体倒序，需要全部移动 1+2+3+4+...+n=n(n+1)/2
平均时间	$O(n^2)$

稳定性	稳！如上图 49 只会和 49 进行比较，比较不小于之后，停在原处即可，不往前继续比较
适用性	顺序表和链表

只有小于之后才会放到前面，所以这里并不会交换位置

YYK
2026/11/05 09:22

链表相对于顺序表来说，扫描的方向，或者说排好序的方向可能是不同的，你自己画个图操作一遍吧

YYK
2026/08/13 10:34

（3）特殊点

07. 在下列算法中，（ ）算法可能出现下列情况：在最后一趟开始之前，所有元素都不在最终位置上。

A. 堆排序　　B. 冒泡排序　　C. 直接插入排序　　D. 快速排序

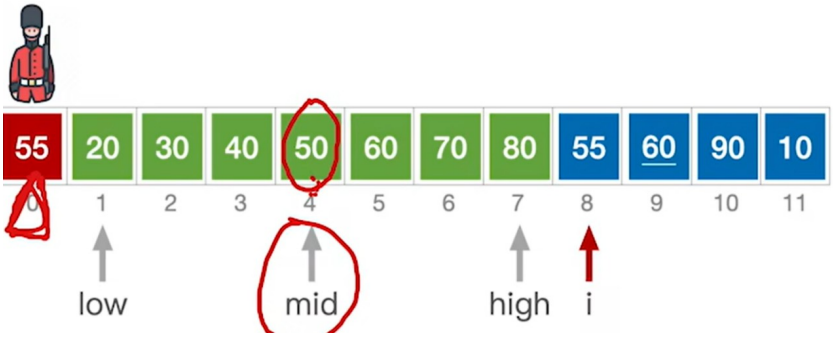
这题自己在做的时候忽略了这个关键的知识点，假设最后一个元素最小，那么在执行最后一趟之前，所有元素都是不在最终位置上的，都需要向后移动一个位置

YYK
2026/08/09 20:16

(三)折半插入排序

（1）算法思想

折半查找有序序列，顺序移动元素



（2）算法性能

空间	只占用常数空间，故为 O（1）
最好时间	整体扫描*每次进行二分查找，故为 O（$n\log_2 n$）
最坏时间	整体倒序，需要全部移动，扫描*移动，故为 O（n^2）
平均时间	O（n^2）
稳定性	稳！如上图 60 只会和 60 进行比较，比较不小于之后，会插入到该数的右半区域，所以不改变位置顺序

这是一个非常有意思的现象，二分查找只改变了查找效率，移动元素的效率仍然没有改变，涉及到移动的话，效率就是 n^2

YYK
2026/08/09 17:56

适用性	顺序表，而不适合链表（涉及到折半查找）

（3）特殊点

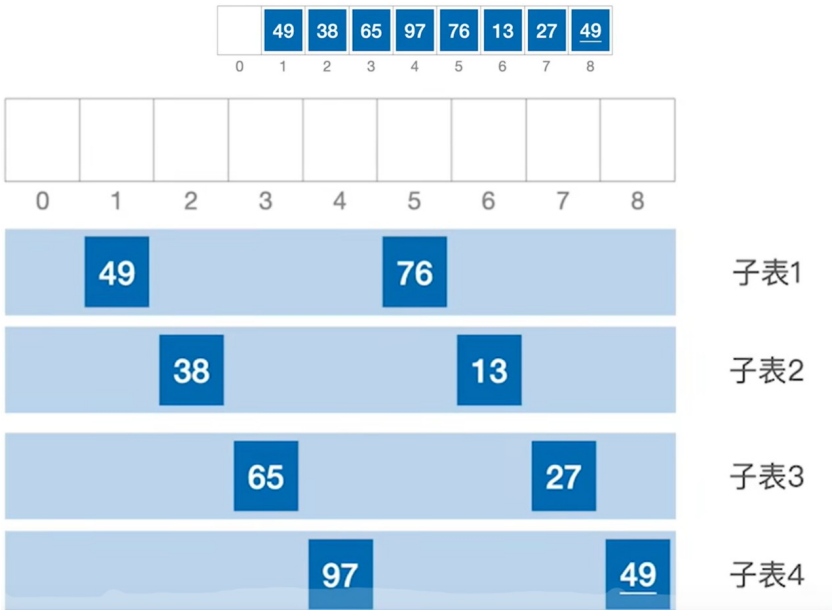
①折半排序每次比较的次数=$\log_2 m$，其中 m 为已经排好序的长度

(四)希尔排序(shell sort)

（1）算法思想

基于插入排序的优化，先追求部分 di（分裂断开的）局部有序，再追求全局有序

希尔排序：先将待排序表分割成若干形如 $L[i, i+d, i+2d, \ldots, i+kd]$ 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量d，重复上述过程，直到d=1为止。



（2）算法性能

空间	只占用常数空间，故为 O（1）
时间复杂度和 d1、d2、d3....的大小有关，无法具体算出，但是最差的时间复	

杂度是 O（n^2）（此时 d1=1，希尔排序退化为直接插入排序） 当 n 在某个范围的时候，时间复杂度约为 O（$n^{1.3}$）	
稳定性	不稳！有时候两个相同的树被划分为两个不同的子表，在子表内部发生调换之后，从整体来看，两个数的顺序也可能发生变换
适用性	顺序表，而不适合链表（涉及到划分子表）

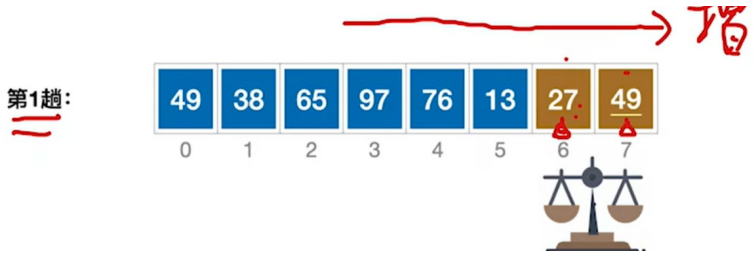
（3）特殊点

①希尔排序组内采用的是直接插入排序

(五)起泡排序(bubble sort)

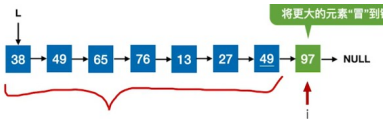
（1）算法思想

基于交换思想，从一侧开始查找，只交换相邻的两个元素，每一趟都需要比较多次、交换多次



（2）算法性能

空间	只占用常数空间，故为 O（1）
最好时间	一趟扫描，设置 flag 如果某一趟没有交换，则数列已经有序、停止排序，故为 O（n）
最坏时间	整体倒序，需要全部移动，扫描*移动=交换次数

	=n-1+n-2+...1=n(n-1)/2，故为 O（n^2）
平均时间	O（n^2）
稳定性	稳！但是稳定的前提是 A[j-1]>A[j],如果加等号，可能就不稳
适用性	顺序表，链表也可以 

注意比较元素的次数和移动元素的次数的区别，每次移动都需要 3 次操作，a、b、temp
YYK
2026/08/10 10:28

（3）特殊点

①每一趟都有一个元素被放置在最终位置上

②移动好位置的元素，以后就不需要再和别的元素进行比较了，例如 5 8 4 6 1，下一次 4 移动到 6 的位置即可了，不需要和 1 比较再确定位置了

③有时候问排序的趟数，题目也不说清楚包不包含最后一趟扫描但无交换元素的那一趟。。

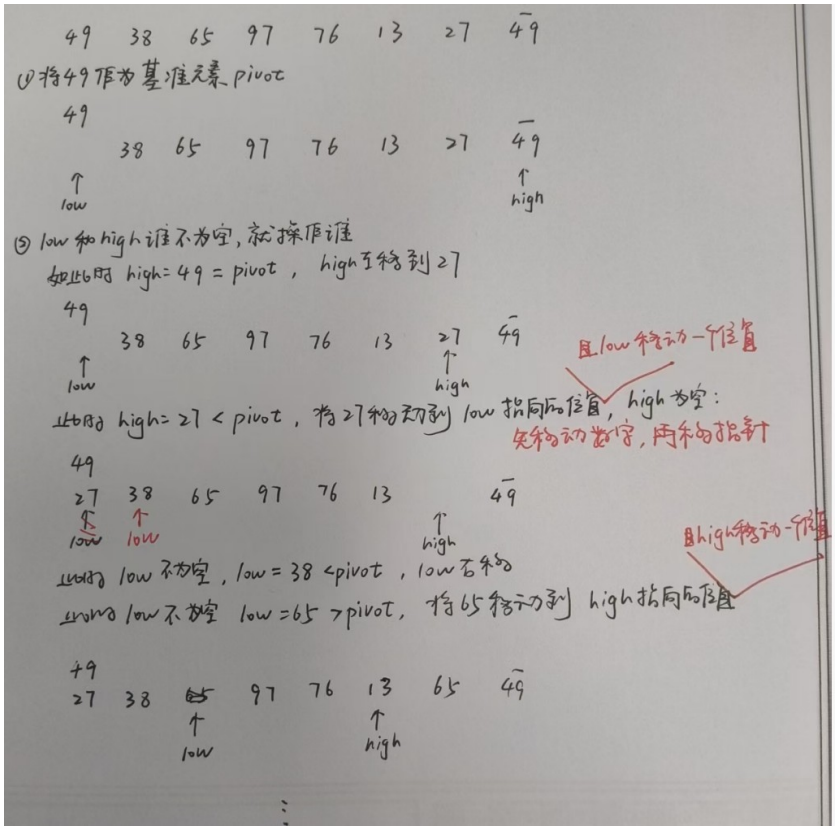
④比较次数与初态有关

(六)快速排序

每趟排序都把大区间划分为左右小区间，小区间之间相对有序
SY
2026/07/15 18:15

（1）算法思想

基于交换思想，从首元素开始作为基准元素，移动 low 和 high 指针，谁不为空就操作谁，直到 high 和 low 相遇！



（2）算法性能

空间	与递归层数 $\log_2 n$ 大致一样，故平均为 O（$\log_2 n$），最坏为 O（n）
最好时间	O（$n*\log_2 n$）
最坏时间	O（n*n层）=O（n^2）
平均时间	O（n*递归层数），靠近于 O（$n*\log_2 n$）
稳定性	不稳！位于同一右边区间的两个元素，先后被移动到左区间，相对位置一定发生了改变，例如5 4 3 2 1 1，由于先扫描的是1，然后再扫描移动了1，所以最终是1 1 2 3 4 5 。

快速排序的最坏的空间复杂度，即有序状态下在进行 n 层递归
YYK
2026/08/12 22:01

这个递归层数要看这个二叉树的形状， n 个节点的二叉树高度最小 $=\log_2 n$ 向下取整 +1 ，最差就是 n 层
YYK
2026/08/10 10:55

判断稳定 / 不稳定，必须有相同关键字
快速排序可能交换相同关键字相对位置
所以找含重复关键字且划分可能跨越枢轴移动的序列
SY
2026/07/15 18:17

适用性	顺序表，不适用于单链表，双链表的话可以考虑一下。

（3）特殊点

①每一趟都至少有一个元素（枢轴元素）被放置在最终位置上

在 408 中的定义，一趟排序包括对左区间和右区间的排序，所以应该是至少有两个枢轴元素被确定该位置
YYK
2026/08/10 11:04

②写代码的时候分两块，一个是递归函数的 Partition，另一个是调用递归函数

```
void QuickSort(int A[],int low,int high){
    if(low<high){    //递归跳出的条件
        int pivotpos=Partition(A,low,high); //划分
        QuickSort(A,low,pivotpos-1);    //划分左子表
        QuickSort(A,pivotpos+1,high);   //划分右子表
    }
}
```

③快速排序的平均性能是最好的

性能关键在于每次划分是否均匀
划分越均匀，越接近 O(n\log n) ；越偏，越接近 O(n^2) 。
SY
2026/07/15 18:13

④快速排序什么时候才能结束！即使第二趟你看着已经排好序了，但是“机器”并不知道，

它需要继续执行完剩下的所有层递归（想清楚，这里指的是什么。假设最终元素是 5 8 9，只是以 8 为枢轴的左右区间分好了，但是左区间内部 5 和右区间内部 9 都需要再遍历一次），才可以！

10. 对下列关键字序列用快速排序进行排序时，每次选取的基准元素都为待处理序列的第一个元素，速度最快的情形是（ ），速度最慢的情形是（ ）。

A. {21, 25, 5, 17, 9, 23, 30}　　B. {25, 23, 30, 17, 21, 5, 9}

C. {21, 9, 17, 30, 25, 23, 5}　　D. {5, 9, 17, 21, 23, 25, 30}

例如本题目，最快的情形就是高度最小的二叉树，最坏的情形就是整体已经全部排好序了，需要 n 层递归！答案是 AD
YYK
2026/08/10 11:36

⑤假设问你下列哪个是第一趟排序之后的元素，例如 6 5 8 2 1 10 本身也是，

因为满足了 10 左边都比 10 小。

(七)简单选择排序

（1）算法思想

基于交换思想，每一趟都找到该躺的最小元素，与该趟的第一个元素进行交换，

n 个元素只需要 n-1 趟扫描

要注意与直接插入排序的区别，直接插入排序虽然也是有序序列 + 无序序列，但在每次比较的过程中可能也发生了“插入”
YYK
2026/08/10 11:46

（2）算法性能

空间	常数个辅助单元 O（1）
时间与初始状态无关，永远都要进行 n(n-1)/2 次扫描，故时间复杂度为 O（n^2） 移动次数最多不超过 3(n-1)次	
稳定性	不稳！因为会发生“隔空的交换”，例如 2 2 1，由于 1 最小，所以要和第一个位置交换，所以变成了 1 2 2，显而易见发生了位置次序的变化
适用性	顺序表，链表，关键字比较少的情况下也可以考虑一下

（3）特殊点

①排序过程中比较次数与序列初始状态无关，永远都要进行 n(n-1)/2 次扫描

(八)堆排序

（1）算法思想

基于交换思想、堆的数据结构、与顺序存储的完全二叉树概念接近，结合复习！

堆的数据结构特性：
①大根堆是根 > 左、右

②小根堆是左、右 > 根
YYK
2026/08/10 14:23

①建立大根堆（一次性先排成二叉树，然后进行调整，而不是一个一个元素加入）、检查该堆是否符合大根堆的特性：

首先找到大根堆中最大编号（n/2 向下取整）的非叶节点，从该节点出发检查所有非叶结点是否满足根>左右，若不满足，则把该结点的最大孩子节点与之互换即可。同时遇到了一个麻烦的问题：把该节点与孩子节点互换之后，导致子树又不满足堆的定义了，这时候代码会继续探测子树是否满足条件，然后不断调整子树。而不是固有思想，让它从最后一个非叶结点再扫描一遍，这是错误的

注意这里是倒序检查，从 7 号、 6 号、 5 号依次检查到根节点，不能跳
YYK
2026/08/10 14:34

```c
//建立大根堆
void BuildMaxHeap(int A[],int len){
    for(int i=len/2;i>0;i--)    //从后往前调整所有非终端结点
        HeadAdjust(A,i,len);
}


//将以 k 为根的子树调整为大根堆
void HeadAdjust(int A[],int k,int len){
    A[0]=A[k];                  //A[0]暂存子树的根结点
    for(int i=2*k;i<=len;i*=2){ //沿key较大的子结点向下筛选
        if(i<len&&A[i]<A[i+1])
            i++;                //取key较大的子结点的下标
        if(A[0]>=A[i])  break;  //筛选结束
        else{
            A[k]=A[i];          //将A[i]调整到双亲结点上
            k=i;                //修改k值，以便继续向下筛选
        }
    }
    A[k]=A[0];                  //被筛选结点的值放入最终位置
}
```

这个代码非常的奇妙，关键在于把 A[i] 调整到双亲节点上，然后又把 i 赋值给 k 了，继续进行对子树的扫描，避免子树因为刚刚的调整而不满足堆的定义，这个过程可以把它叫做“下坠”

每次下坠，最多比较两次关键字，第一次是比较左右孩子（有可能没有左右孩子，所以这一步可能不存在）、第二次是比较该节点与其最大的孩子节点。

若该树高 h ，该节点位于第 i 层，则最多进行 h-i 次下坠，最多进行 2 （ h-i ）次比较

YYK
2026/08/10 14:42

②基于大根堆进行排序

首先是把末尾元素与根节点（含有最大值）进行互换，删除交换之后的末尾节点（相当于取出了最大值），然后继续调整大根堆使之满足大根堆的定义

```c
//堆排序的完整逻辑
void HeapSort(int A[],int len){
    BuildMaxHeap(A,len);        //初始建堆
    for(int i=len;i>1;i--){     //n-1趟的交换和建堆过程
        swap(A[i],A[1]);        //堆顶元素和堆底元素交换
        HeadAdjust(A,1,i-1);    //把剩余的待排序元素整理成堆
    }
}
```

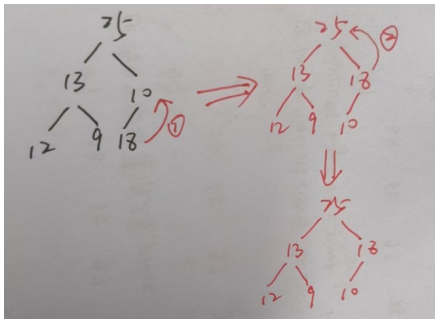
这一步不需要像构建大根堆那样，从最后一个非叶节点开始向前遍历是否符合大根堆定义，此时只需要将根节点的值、 len-1 （因为刚刚删除了一个元素）传入 HeadAdjust 函数，让它调整即可

YYK
2026/08/10 14:46

③大根堆的插入算法

首先把元素插在最后一个元素的位置上，然后调整大根堆的结构使之满足定义。

这里要区分于基于大根堆进行排序的算法，这里只需要与父节点进行对比即可，不需要对兄弟节点进行对比，因为兄弟节点一定小于父节点。（2011 真题）



④小根堆的删除算法（2015 真题）

（2）算法性能

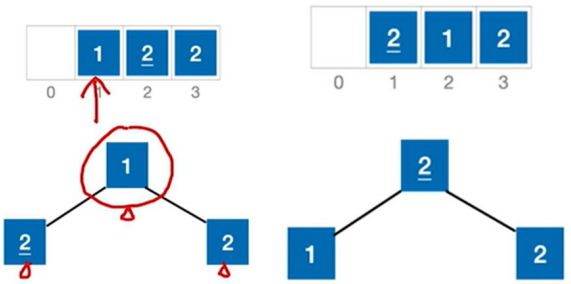
空间	常数空间 O（1）
建立大根堆	比较次数最多为 4n 次,时间复杂度 O（n）
每个元素调整	根节点与末尾节点互换后（取数操作），调整根节点最多进行 h-1 层下坠，即最多进行 2（h-1）次操作，所以每次调整时间复杂度为 O（h）=O（$\log_2 n$）
所有元素调整	想要取出所有数，即需要进行 n-1 次取数操作，故时间复杂度为 O（$n*\log_2 n$）
插入元素	向上调整的算法，比较次数最多为树的高度-1，即 $\log_2 n$ 向下取整
删除元素	向下调整的算法，比较次数最多为树的高度-1，即 $\log_2 n$ 向下取整
稳定性	不稳！下面这个例子原来的顺序是 1 2 2，但是在经过大根堆的定义优化之后变成了第二张图

与每个元素调整的理念一样

YYK
2026/08/10 20:06

请注意这里的逻辑变化，是先将左孩子与右孩子比较，如果左孩子小于右孩子，才会取走右孩子与根节点互换，但是如果等于呢？这里的逻辑是根节点优先和左孩子互换。

YYK
2026/08/10 15:21

	 接下来进行取数操作，可以知道第一个取出的数（最大数）一定是根节点 2，那么后续的才会取出 2，显然排序顺序变成了 1 2 2（顺序发生了变化，不稳定！）
适用性	顺序表，不适用于单链表，双链表的话可以考虑一下。

关键是找父亲节点太复杂了、找孩子节点也很复杂

YYK
2026/08/13 10:36

（3）特殊点

①适用于在海量数据中找到前 xx 位数据

②堆排序可以分别分左右子树同时并行执行，互不干扰！

25．将关键字 7,5,1,4,6,8,9 依次插入初始为空的小根堆 H，得到的 H 是（　）。

A．1,4,5,7,6,8,9　　B．1,4,7,5,6,8,9　　C．1,4,5,6,7,8,9　　D．1,5,4,6,8,9,7

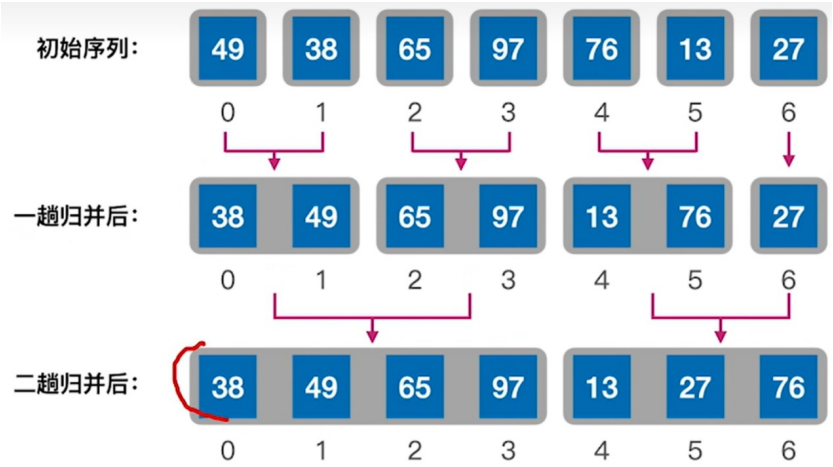
依次插入：
7
7，5 结果发现 7 大于 5，所以顺序是 5，7
5，7，1，结果发现 5 大于 1，所以顺序是 1，7，5
以此类推即可获得答案 A
这种题目是固定的，1475689 虽然也符合小根堆，但是不符合依次插入喵

SY
2026/07/15 18:28

(九)二路归并排序(merge sort)

（1）算法思想

每次选择相邻的两个有序分组，进行归并



（2）算法性能

空间	与原数组空间大小一样，故为 O（n）（主要用于复制数组，否则没地方存储）
最好时间	O（$n*\log_2 n$）
最坏时间	O（$n*\log_2 n$）
平均时间	O（n*递归层数）故为 O（$n*\log_2 n$）
稳定性	稳定！如果左右两个区间出现了相同的数，我们会优先让左边的数进入排序好的数组，所以相对顺序不变
适用性	顺序表、链表

如果是 k 路归并的话，那么只需要把下标 2 修改为 k 即可

YYK
2026/08/12 16:31

（3）特殊点

①归并排序本质上也是递归算法，需要借助外部空间 n 个单位用来临时存储元素，设置两个指针 i 和 j 指向左右两个数组的第一个元素

```
int *B=(int *)malloc(n*sizeof(int)); //辅助数组B

//A[low…mid]和A[mid+1…high]各自有序，将两个部分归并
void Merge(int A[],int low,int mid,int high){
    int i,j,k;
    for(k=low;k<=high;k++)
        B[k]=A[k];           //将A中所有元素复制到B中
    for(i=low,j=mid+1,k=i;i<=mid&&j<=high;k++){
        if(B[i]<=B[j])
            A[k]=B[i++];     //将较小值复制到A中
        else
            A[k]=B[j++];
    }//for
    while(i<=mid)   A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}

void MergeSort(int A[],int low,int high){
    if(low<high){
        int mid=(low+high)/2;    //从中间划分
        MergeSort(A,low,mid);    //对左半部分归并排序
        MergeSort(A,mid+1,high);//对右半部分归并排序
        Merge(A,low,mid,high);  //归并
    }//if
}
```

②关于时间复杂度的分析

二叉树的第h层最多有 2^{h-1}个结点

若树高为h，则应满足 $n \leq 2^{h-1}$

即 $h-1=\lceil log_2 n \rceil$

结论：n个元素进行2路归并排序，归并趟数=$\lceil log_2 n \rceil$

每趟归并时间复杂度为 $O(n)$，则算法时间复杂度为 $O(nlog_2 n)$

③归并排序的代码比直接插入排序复杂得多（2017 真题）

④二路归并的比较次数为 n+m-1，两个数组的个数分别为 n 和 m

这也是两个升序数组比较的最多的情况， 2012 真题

YYK
2026/08/18 21:27

(十)基数排序

（1）算法思想

基数排序有 LSD 和 MSD 两种， LSD 是指最低位优先，第一趟的比较结果是个位，第二趟虽然比较的是十位，但是实际收集之后，保证的是十位拼接个位是保证有序的！！！单独的十位甚至可能是无序的

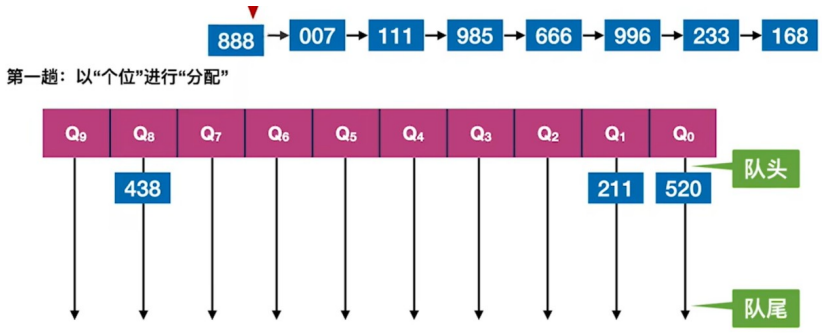
YYK
2026/08/12 22:10

是对个位元素、十位元素、百位元素依次进行对比：

首先对个位元素依次放入对应的“链表中”，全部放入后，从每个链表中取下数据

然后对十位元素依次放入对应的“链表中”.....

其实本质上每一趟对个位、十位+个位、百位+十位+个位进行了排序，最终得到想要的结果



（2）算法性能

空间	O（r），r 个队列、r 个队尾和队首指针
一趟排序	O（n），扫描整个数组元素
一趟收集	O（r），这个 r 其实就代表 10 个队列，每个队列首尾相接即可，只需要 O（1）,所以总共是 O（r）
全部时间	关键看元素中总共有 d位，完整的一趟=排序+收集，总共需要 d 趟，所以时间是 O（d*(n+r)）

也可以适用于其他的，比如出生年月日，就可以划分为 3 个位就行

YYK
2026/08/12 16:50

基数排序要按 排，最多排 d
每一趟都要做一次分配和收集
一趟的代价是：
扫一遍 n 个关键字
再处理 r 个桶
所以是 O(d(n+r))

SY
2026/07/16 14:25

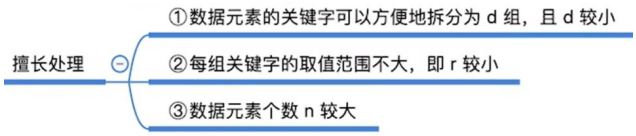
稳定性	稳定！
适用性	顺序表、链表

（3）特殊点

①基数排序不是基于“比较”的算法

②与数据的初始状态无关

③擅长的应用



④实数包含整数、float 和 double，显然针对实数来说，基数排序并不合适。基数排序只能针对于整数

34．已知进行 2-路归并排序的待排序记录的关键字当前序列为{49, 38, 65, 97, 76, 13, 27}，下列说法错误的是（　）

A．该序列只可能是初始状态　　B．要进行 3 趟归并

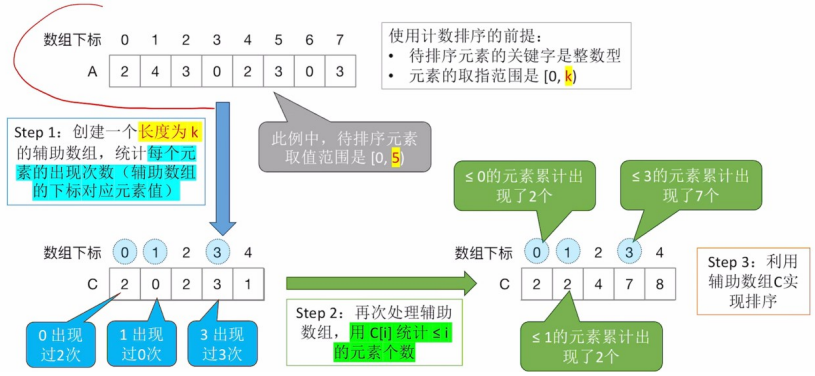
C．第一趟关键字比较的次数是 4 次　　D．第二趟归并后，13 的相邻元素是 97，27

A 选项是对的，因为如果不是初始状态，肯定有相邻两个是有序的
SY
2026/07/15 18:39

(十一)计数排序

（1）算法思想

不基于比较的排序算法，开辟一个数组 B 和数组 C



（2）算法性能

空间	O（n+k）,n 代表开辟的 n 个元素空间，k 代表开辟的 0-k 的范围数组即 C 数组
两趟扫描 C 数组	O（2k），第一次扫描是为了把 C 数组初始化为 0，第二次扫描是为了要统计比该数小于等于的数有多少个，即 C[i]=C[i]+C[i-1]
两趟扫描 A 数组	O（2n），第一次扫描是为了确定 C 的初始值，第二次扫描是为了根据 C 的初始值来确定 A 的每个元素的位置
全部时间	O（2n+2k），即 O（n+k）
稳定性	稳定！
适用性	更适合整数+顺序表

这个初始值只是记录了每个数出现了多少次，但是数组 C 最终的结果是要统计比该数小于等于的数有多少个，即 C[i]=C[i]+C[i-1]
YYK
2026/08/12 20:28

当 k=O（n）时，时间复杂度最好为 O（n），但是如果 k>O（n*log2n），那么该算法比快速排序和堆排序还要差了
YYK
2026/08/12 20:30

（3）特殊点

①代码




```
// 计数排序
void CountSort(int A[], int B[], int n, int k){
  int i,C[k];              //辅助数组C的长度取决于待排序元素取值范围[0, k)
  for(i=0;i<k;i++)         //初始化计数数组
    C[i]=0;
  for(i=0;i<n;i++)         //Step1：遍历待排序数组，统计每个关键字的出现次数
    C[A[i]]++;
  for(i=1;i<k;i++)         //Step2：再次处理辅助数组，统计不大于 i 的元素个数
    C[i]=C[i]+C[i-1];      //C[i]保存的是小于或等于 i 的元素个数
  for(i=n-1;i>=0;i--){     //Step3：利用辅助数组C实现计数排序（从后往前处理）
    C[A[i]]=C[A[i]]-1;
    B[C[A[i]]]=A[i];   //将元素 A[i]放在输出数组 B[]的正确位置上
  }
}
```

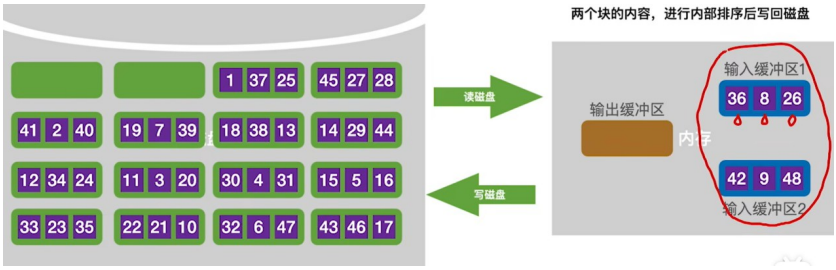
(十二)外部排序

1.外部排序的方法

(1)构造初始化归并段

外存的所有“数据块”依次读两个数据块入内存，在内存中对其进行归并排序，

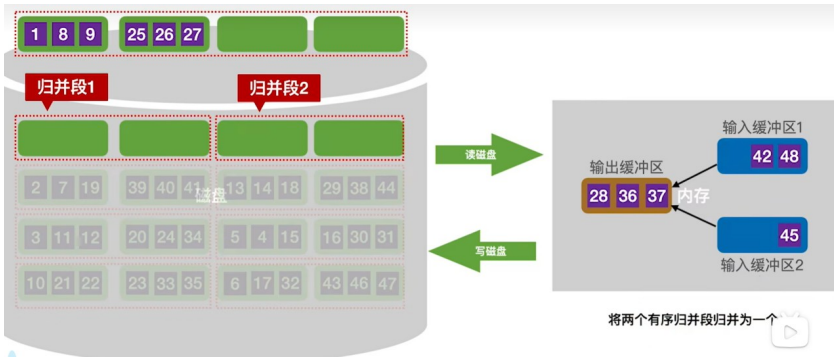
使得两个数据块内部、外部是有序的！





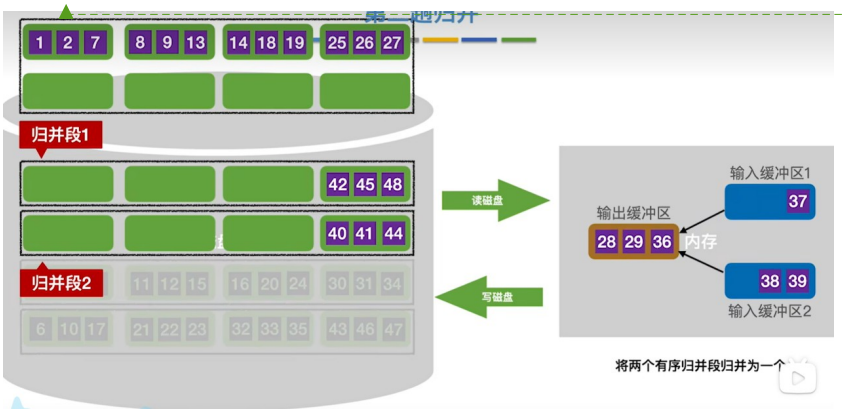
（2）第一趟归并

分别取出两个归并段的第一个数据块，然后依次取出前三个数据，如果某个数据块被取空了，那么需要从该数据块所属的归并段中取出下一个数据块，直到所有数据块全都被取空



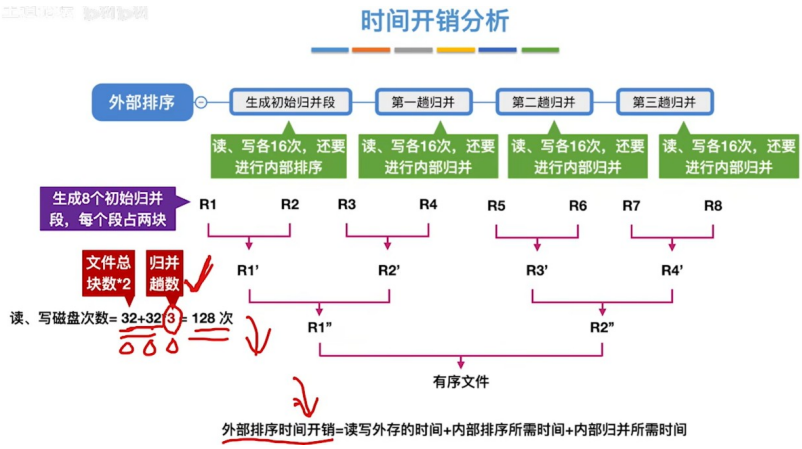
（3）第二趟归并

同理：



一定要注意的是，如果输入缓冲区 1 空了，一定要先拿出归并段中的一个数据块填补空缺，才能继续比较两个缓冲区的大小
YYK
2026/08/13 12:47

（4）时间开销分析



所以，读写外存的时间占据了时间开销的绝大大大多数部分

YYK
2026/08/13 14:39

（5）优化思路（减少归并趟数）

多路归并，把二路归并优化为四路归并||||或者减少初始归并段的数量

多路归并带来的负面影响：
①k路归并时，需要开辟k个输入缓冲区，内存开销增加。
②每挑选一个关键字需要对比关键字（k-1）次，内部归并所需时间增加

这是基于多路归并的缺点：归并路数越多，每次的关键字比较次数也会增多。例如有 1024 个归并段， 1024 路归并每次就需要比较 1023 次关键字对比

败者树基于缺点进行了优化，基于败者树，每次只需要比较很少的次数 ----- 树的层高

YYK
2026/08/13 12:54

2.败者树

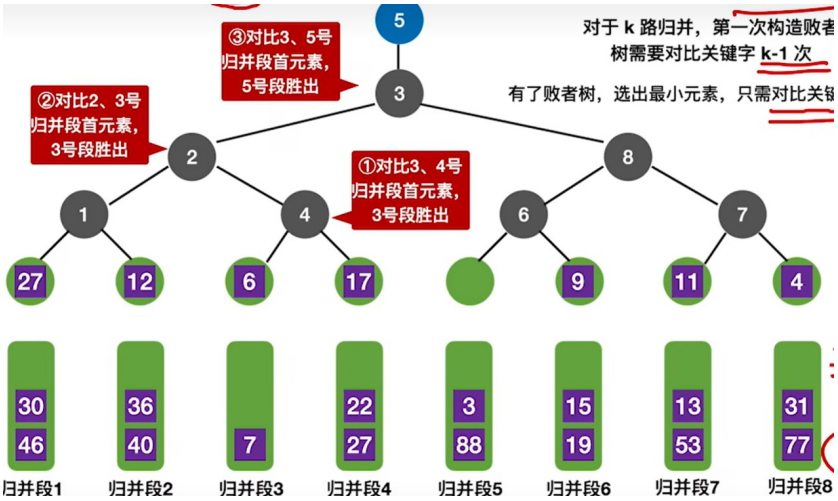
（1）定义

①可视为一棵完全二叉树，最高处的“胜者”不代表树的层高

② 有了败者树，选出最小元素，只需对比关键字$\lceil log_2 k \rceil$次

与败者树的层数有息息相关的联系，这里的关键字对比是最多 log2k 次，有可能是少于这个次数的

YYK
2026/08/13 13:00



这里面圆形存储的是归并段的编号

3.置换-选择排序

这是基于多路归并的缺点：初始归并段太多。

置换选择排序基于缺点进行了优化，可以很好的减少初始归并段的数量

YYK
2026/08/13 13:06

（1）问题分析

假设有 24 个数据，但是内存工作区只有 3 个元素大小，那么就至少要分 8 个归并段。但是引入置换选择排序就可以很好减少归并段数量

（2）算法



每次输出内存工作区中最小的元素，同时记录归并段中最大的元素。内存工作区中无法放倒归并段后面的数标记为红色，表明它一定属于第二个归并段的，直到内存工作区全都变成红色，表明需要新开辟一个归并段了

这个是需要借助内存输入输出缓冲区来完成的，每次读入的新的数据块、即将整理好的归并段都存在缓冲区中

YYK
2026/08/13 13:15

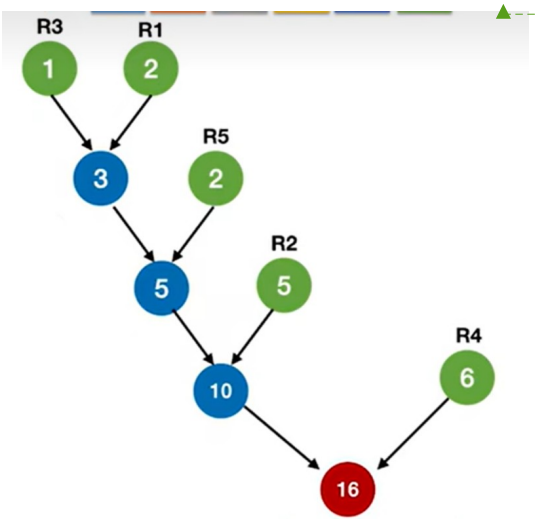


4.最佳归并树

归并过程中 IO 磁盘的次数=归并树带权路径长 WPL*2

所以二路归并的最佳归并树就是一个倒过来的哈夫曼树

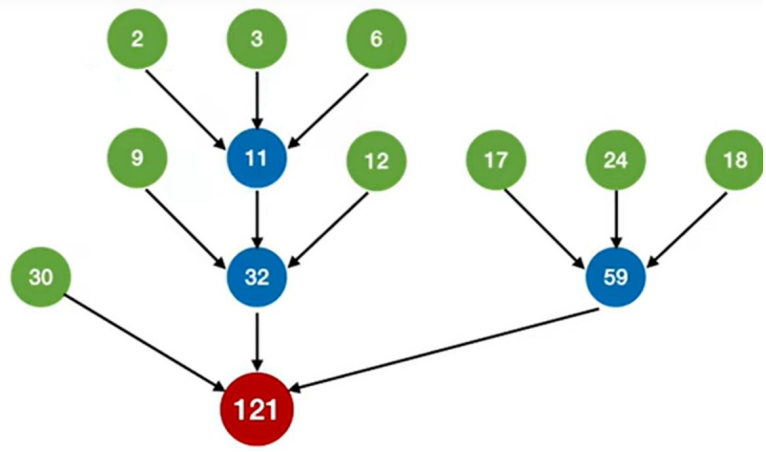
YYK
2026/08/13 14:31



多路归并的最佳归并数与哈夫曼树非常类似，都是选择最小的三个节点成为兄弟

但对于 k 叉归并来说，如果某次归并不满足 k 个元素相加，那么就需要添加 0 号虚拟归并段

YYK
2026/08/13 14:34



69．设外存上有 80 个归并段，进行 8 路平衡归并，为实现最佳归并，需要补充的虚段个数是（　）

A．4　　B．5　　C．6　　D．不需要补充

写规范一点是：

$$(80+x) \bmod 7 = 1$$

因为：

$$80 \bmod 7 = 3$$

所以变成：

$$(3+x) \bmod 7 = 1$$

要让 `3 + x` 除以 7 余 1，最小的非负 `x` 是 5：

为什么 mod7 ，因为本来是 8 路归并，要最佳归并，就归并为 1 路，所以会是 mod（ 8-1 ），为什么最后要等于一，就是因为归并段 mod7 后，只有一路！
SY
2026/07/15 15:42

(十三)排序算法的分析和应用

表 8.1　各种排序算法的性质

算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序				$O(1)$	否
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	否
二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是

基于比较的排序，最坏情况下有下界
也就是不可能普遍低于 nlogn 。
快速排序、归并排序、堆排序这些都绕不开比较模型下界。
SY
2026/07/16 14:29

排序与初始状态的关系：
插入排序：和初始状态有关。越接近有序越快。
快速排序：和初始状态有关。划分是否均匀影响很大。
选择排序：基本无关。每一趟都要从剩余元素里找最小值。
归并排序：基本无关。都是不断二路归并，时间稳定是 O(nlog n) 。
SY
2026/07/16 14:31

小结：

不稳定排序算法：简希快堆！

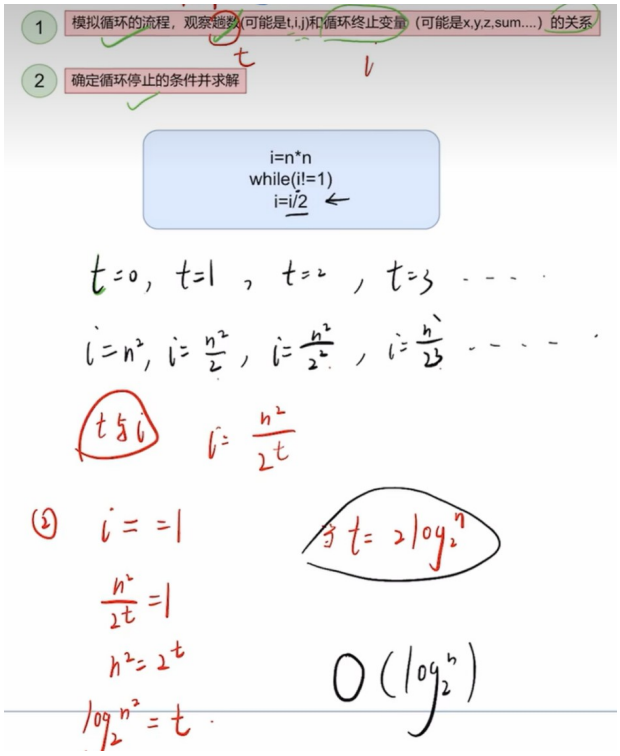
只用顺序表：折希快堆！

元素个数少（数据小于 10000）：直接插入排序、简单选择排序、冒泡排序

元素个数多：快速排序、堆排序、归并排序

基本有序：冒泡、直接插入

时间复杂度


1 模拟循环的流程，观察变量(可能是t,i,j)和循环终止变量（可能是x,y,z,sum.....）的关系
2 确定循环停止的条件并求解
i=n*n
while(i!=1)
i=i/2
t=0, t=1, t=2, t=3 ....
i=n², i=n²/2, i=n²/2², i=n²/2³ ....
t与i
i=n²/2^t
② i==1
n²/2^t=1
n²=2^t
log₂n²=t
则 t=2log₂n
O(log₂n)

找趟数和循环终止条件之间的关系：

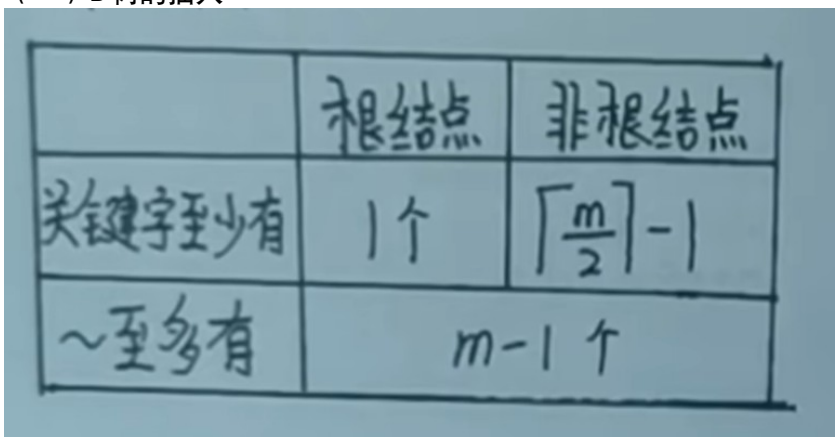
趟数 t=0 t=1 t=2 t=3 .....

关键是要计算运行的次数之和和规模之间的关系

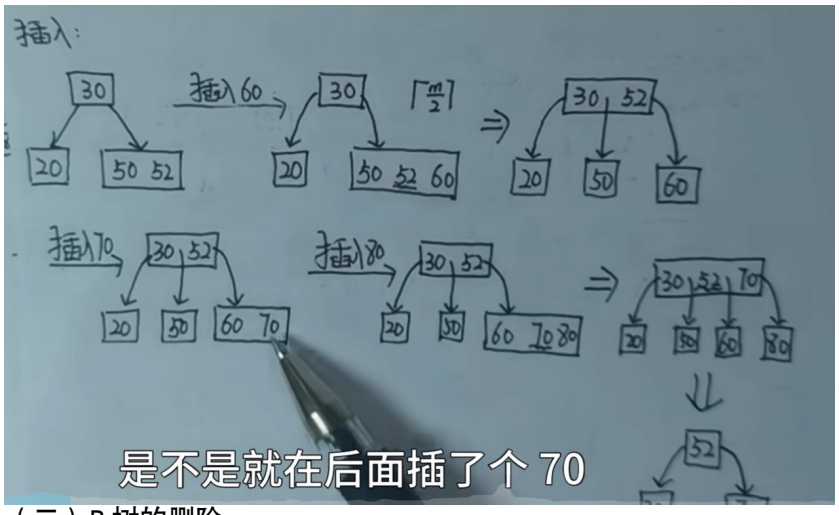
找执行趟数 t 和中间变量 x 的关系，然后映射到最终的规模 n 上面

八、B 树的插入与删除

（一）B 树的插入



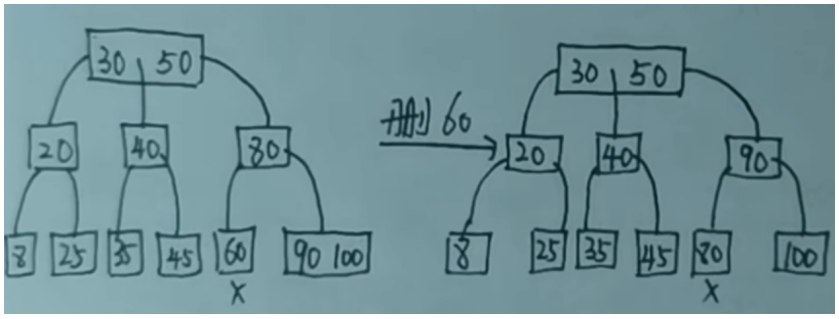
B 树先直接插入，如果数量大于要求的关键字数量，那就取出其中 M/2 向上取整该位置的数，向上调整到上一层



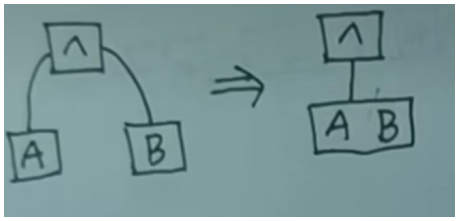
（二）B 树的删除

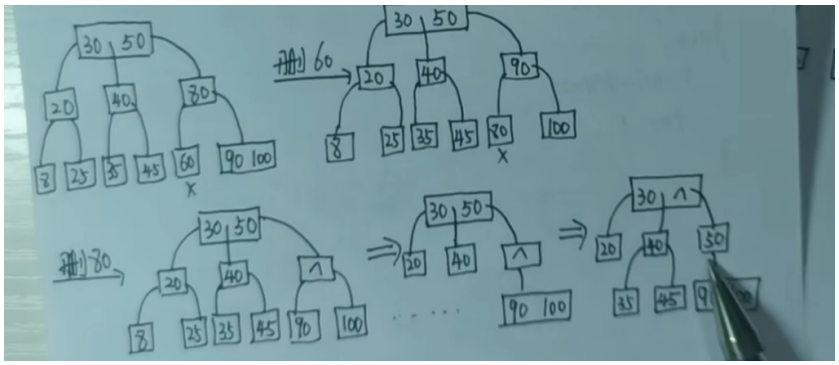
（1）删除后，条件仍然满足，不需要变动

（2）删除后，先找兄弟借，借到了，让兄弟的节点上去当父节点，父节点下来当孩子节点



（3）删除后





这个合并的太牛了，先将父节点拉下水，然后合并子节点，继续将父父节点拉下水，然后继续合并

（4）B 树的规定

14.【2013 统考真题】在一棵高度为 2 的 5 阶 B 树中，所含关键字的个数至少是(　　)。

A.5　　B.7　　C.8　　D.14

